



2025 年（第 18 届） 中国大学生计算机设计大赛

人工智能实践赛作品报告

作品编号：_____2025074585_____

作品名称：_____ImageGuard_____

_____基于 MPC 的图像内容隐私保护系统_____

填写日期：_____2025 年 5 月 1 日_____

填写说明：

- 1、本文档适用于人工智能挑战赛预选赛；
- 2、尽管预选赛仅完成部分工作，但是本文档需要针对决赛做出方案设计；
- 3、正文、标题格式已经在本文中设定，请勿修改；标题#的快捷键为“Ctrl+#”，正文快捷键为“Ctrl+0”；
- 4、本文档应结构清晰，突出重点，适当配合图表，描述准确，不易冗长拖沓；
- 5、提交文档时，以 PDF 格式提交；
- 6、本文档内容是正式参赛内容的组成部分，务必真实填写。如不属实，将导致奖项等级降低甚至终止本作品参加比赛。

目录

1	作品概述	1
1.1	研究意义	1
1.2	相关工作	3
1.3	创新性	4
1.4	应用背景	5
1.5	综合价值	6
2	作品介绍	7
2.1	系统框架	7
2.2	系统模块	9
2.2.1	用户管理模块	9
2.2.2	数据管理模块	10
2.2.3	项目管理模块	10
2.2.4	系统日志模块	11
2.2.5	秘密共享模块	11
2.2.6	数据隐藏模块	12
2.2.7	参数生成模块	13
2.2.8	数据交换模块	14
2.2.9	神经网络训练模块	14
3	实现方案	16
3.1	开发环境	16
3.1.1	设备配置	16
3.1.2	开发工具	16
3.2	Instahide 的实现方案	17
3.2.1	秘密分享	17
3.2.2	双方 Instahide 协议	19
3.3	参数生成模块	20
3.3.1	Beaver 乘法三元组	21
3.3.2	参数生成模块主要运算	21
3.4	双服务器神经网络训练框架的实现方案	24
3.4.1	双服务器神经网络模型	25
3.4.2	协助算法	25
3.4.3	前向传播过程	27
3.4.4	反向传播过程	30
3.5	系统基础模块的实现方案	32
3.5.1	用户管理模块	32
3.5.2	数据管理模块	33

3.5.3	项目管理模块	33
3.5.4	系统日志模块	34
3.6	数据交换模块	35
4	系统测试	37
4.1	测试环境	37
4.2	系统功能性测试	37
4.2.1	数据隐藏效果测试	37
4.2.2	数据隐藏性能测试	38
4.2.3	模型训练效果测试	40
4.2.4	COT、同态加密计算速度比较	40
4.2.5	模型训练速度测试	41
4.3	系统安全性测试	42
4.4	系统完整性测试	43
4.4.1	用户注册测试	44
4.4.2	用户登录测试	44
4.4.3	用户管理测试	45
4.4.4	项目管理测试	46
4.4.5	日志模块测试	48
5	创新性	49
5.1	赋能数据安全	49
5.2	提供可靠算法	49
5.3	打通数据孤岛	50
5.4	分割数据权属	50
6	总结	51

摘要

随着信息技术的飞速发展，数据已然成为重要的生产要素，成为提升企业竞争力、推动经济发展质量变革的重要引擎。然而，在企业、机构等对于个人隐私信息数据使用的背后，个人、国家对于数据隐私保护的担忧与顾虑日益增加，数据能否仅被用于协同训练而不会被其他用户或外部的恶意攻击者窃取，将成为影响数据授权意愿的关键因素。在国家不断加强数据监管的当下，如何保障数据安全与隐私合规，成为数据市场健康发展的核心问题。

在神经网络进行训练的过程中，现存的训练系统存在如下隐私问题：

首先，常规神经网络训练过程的安全性较脆弱，经实验证明，恶意攻击者不但能够通过最后的训练结果轻易推导出训练数据，甚至可以**仅通过获取在训练过程中传递的梯度参数，即对训练者的原始训练数据进行反向恢复**。这也意味着对于用户隐私数据泄露的风险不仅存在于常见的数据库攻击风险中，还存在于研究工作极少关注到的还存在于正常的模型训练过程中。其次，在半诚信行为模型下，涉及到重要的个人隐私数据，数据用户之间互相防备、无法共同参与训练，从而影响正常生产与服务提

针对当前数据共享的安全威胁，本系统拟采用数据实例隐藏和分布式训练的思想设计安全可靠的数据协同训练系统。数据的 InstaHide 实例隐藏，通过综合运用 Mixup 数据增强的数据混合以及掩码处理等技术，在进行训练前对所有用户传入的原始数据进行简单、高效的加密，确保了在神经网络训练的过程中，即使数据被攻击者窃取、恢复，也无法还原出原始数据。分布式训练，则是针对单服务器安全性薄弱，一旦被攻击危害较大的问题，提出的基于秘密共享算法的双服务器训练模型。该模型允许数据所有者将数据以秘密份额的形式在两个非协作的服务器上进行特征提取以及模型训练，在保证训练准确性的同时，显著提高了训练过程的数据安全性。

在测试阶段，本系统主要对训练前的原始数据隐藏效果，训练中的性能开销和速度，训练后得到的模型效果以及训练过程的安全性等方面进行了测试。测试结果表明，本系统提出的基于 InstaHide 实例隐藏的数据混合和处理可以很好地对原始数据进行加密，难以被恢复还原，有效地保证了用户隐私数据的安全性，而双服务器训练模型，极大地改善了服务器训练过程的安全性，且利用 GPU 加速，将多方安全计算带来的时间

性能损失将至尽可能低，将训练场景成功地应用到了多方训练的半诚信行为模型中，证实了我们系统的落地可行性。

本系统创新主要体现在以下几个方面：

(1) 在**数据安全性**方面，本系统采用数据实例隐藏技术，对用户传入的图像数据进行加密存储。针对现存商业系统中以可视形式使用用户本地平台上传的隐私图像数据所带来训练图像隐私泄露隐患，本系统结合多方安全计算技术与 Instahide 算法对图像进行实例隐藏，用户本地平台以秘密数据份额形式将隐私图像上传至服务器，**服务器中不存在任何可以反向恢复原始图像的信息**。从而保证即使分布式服务器之一受到攻击，也不会造成用户数据的泄露。同时利用 GPU 加速，降低多方安全计算即使时间额外开销。

(2) 在**防御对象及场景**方面，不同于传统 MPC 多源域协同训练系统防止系统训练多方互通隐私数据，本系统针对**单数据源域训练过程中，恶意第三方**利用难以避免的服务器漏洞窃取到的训练梯度，反向恢复原始数据的问题，引入广泛应用于多源域协同训练场景的 MPC 技术进行双服务器分布训练部署，结合 Instahide 算法，确保训练数据不可反向恢复。由于现有工作针对训练梯度反向恢复原始数据的防御研究极少，本系统隐私份额与加密技术的结合能够为相关研究工作提供思路启发。

(3) 在**数据共享**方面，由于系统项目包含邀请模块，不仅能够支持单源域数据防反向恢复安全训练功能，也能够支持多方数据所有者利用非协作服务器进行模型协作训练。利用秘密共享技术完成双服务器交互，同时保证神经网络的准确性和数据的隐私性，打通了数据孤岛，让原本不敢被共享的数据得以流通打破数据壁垒，赋能多方协作、驱动业务。

(4) 在**国家数据治理**方面，为数据权属规则制定提供有效解决思路。本系统确保用户隐私数据仅存在本地，服务提供商对用户数据可用而不可知，并且提高了在单一服务器被攻击的情况下仍然不会造成用户数据泄露的安全性，实现数据确权。用户与服务商之间，医药系统与深度模型训练提供者之间，都能够在不交易数据本身的情况下实现数据价值。中。其次，在半诚信行为模型下，涉及到重要的个人隐私数据，数据用户之间互相防备、无法共同参与训练，从而影响正常生产与服务提供。

关键词：隐私图像反向恢复，多方安全计算，Instahide 算法，神经网络，分布式训练

第一章 作品概述

1.1 研究意义

伴随着互联网的迅猛发展和数字经济的快速推进，全球数据呈现爆发增长、海量集聚的特点，对经济发展、社会治理、国家管理、人民生活都产生了重大影响。

作为前沿技术开发、隐私安全保护的重要内容，数据的重要性被提到了前所未有的高度。2020年4月，中共中央、国务院发布《关于构建更加完善的要素市场化配置体制机制的意见》，将**数据**同土地、劳动力、资本、技术等传统生产要素并列，作为一种新型生产要素参与分配。作为重要的生产要素之一，数据已经成为推动经济发展质量变革，是加快经济社会发展质量变革、效率变革、动力变革的重要引擎。随着《数据安全法》《个人信息保护法》等法律法规的逐步出台实施，我国数据市场进入了合规发展的新阶段，数据隐私保护成为重要共识，对信息滥用与泄露的监管日趋严格。

事实上，我国也一直致力于出台各种**政策**以保护数据隐私。2004年出台的《居民身份证法》拉开了我国个人信息保护法制建设的序幕。2017年6月1日实施的《中华人民共和国网络安全法》明确了个人信息等基本概念，并明确表示严厉打击损害数据安全的犯罪活动。2019年1月1日生效的《电子商务法》直接将电子商务中用户个人信息和隐私信息的保护列入法律条款，并作为立法的基础。2021年11月1日，《中华人民共和国个人信息保护法》正式实施。以上均说明了数据安全问题的严峻性和我国政府对此的重视。

作为数据隐私的重要课题之一，**图像隐私保护**在海量应用场景中也面临着**巨大挑战**。其中，人脸识别技术在身份识别、反欺诈、线下支付、智慧网点等场景中被大量使用，但通过攻击平台以牟利的黑灰产也应运而生，不法分子利用盗取的人脸数据，绕过前端活体检测模块，导致人脸识别系统失效，保护人脸图片安全已成为数据隐私保护的重中之重。此外，在远程医疗逐步发展这一大背景下，越来越多的医学图像通过网络进行传输和保存，这些医学图像一旦上传至网络，也就意味着患者失去了对这些数据的掌控，这一现象更是加剧了患者隐私信息泄露的风险。近期，德国一家安全公司披露网络上目前存在人人可得的大量的医学图像，总量或许已经不止十亿级别，对患者的隐私造成了极大威胁。因此为了有效地保护患者医疗图像上相关的个人信息，让患者可以放心地通过远程医疗进行就诊，医疗图像的隐私保护也是目前亟需解决的问题。

目前，对于用户图像隐私数据的保护主要有三个手段，即国家政策的约束与数据滥用现象的整治，数据拥有者构建安全可信的系统抵御外部攻击，以及系统层限制数据的采集。

首先是通过政策的制定来保护用户的图像隐私数据。目前信息安全部门已经陆续出台了各种相关法律法规，2021 年 7 月出台的《最高人民法院关于审理使用人脸识别技术处理个人信息相关民事案件适用法律若干问题的规定》明确提出，社区在使用人脸识别门禁系统录入人脸信息时，应当征得业主或物业使用人的同意，对于不同意的业主，小区物业应当提供替代性验证方式，不得侵害业主或物业使用人的人格权益和其他合法权益。《中华人民共和国个人信息保护法》第十三条指出除非公共卫生事件和紧急事情等情况，处理个人信息应当取得个人同意。数据管理方需要依循法律规定的路径，在具体使用技术的过程中严格遵守数据的授权范围和授权目的，采用必要手段确保数据传输和存储安全，以及为信息废弃制定适当的处置程序和安全保障措施。但即使制定了如此多的相关政策，图像隐私泄露问题仍然是一个难以整治的问题。

另一方面就是通过构建安全的系统来保护用户的图像隐私数据。但攻击和防御往往是相互促进的，即使是大企业，其安全的系统也只能被认定为在几年范围内是安全的，其安全系统也是时刻面临安全威胁的。比如 2011 年 12 月，CSDN 的安全系统遭到黑客攻击，600 万用户的登录名、密码及邮箱遭到泄漏。随后，CSDN" 密码外泄门" 持续发酵，天涯、世纪佳缘等网站相继被曝用户数据遭泄密。苹果公司 iCloud 于 2014 年遭遇黑客攻击，泄露了大量的名人照片。2018 年 9 月 Facebook 爆出，因安全系统漏洞而遭受黑客攻击，导致 3000 万用户信息泄露，敏感信息包括：姓名、联系方式、搜索记录、登陆位置等。可见即使是大公司的安全系统，也无法做到百分百保证用户隐私的安全。

最后一方面是在系统层减少 App 对于用户数据的采集。315 晚会关注儿童智能手表乱象，各种 App 安装后，无需用户授权就可以开启多种敏感权限，轻易获得孩子的位置、人脸图像、录音等隐私信息，孩子的安全隐患可想而知。而大多数苹果设备上包含用户不想暴露给 App 或外部实体的个人数据。如果 App 对数据的使用或访问不当，用户可能会弃用甚至从设备中删除该 App。App 仅在根据适用法律获得用户的知情同意后，才应访问用户或设备数据。此外，App 也应该采取适当的步骤来保护用户和设备数据，并对数据的使用保持透明。App 应该请求并使用完成给定任务所需的最少量用户或设备数据。苹果设备系统禁止 App 出于不必要或不明显的原因，或者认为以后可能会有用而试图获取或收集数据。限制图像数据的采集虽然可以在一定程度上实现对用户

隐私的保护，但也牺牲了很多依赖图像数据的应用。

针对现有图像数据隐私保护技术的弊端，我们提出的图像隐私计算以第三代人工智能技术为依托，利用多方安全计算、联邦学习、可信执行环境等隐私保护计算技术打造的数据安全计算基础设施。我们提出的图像隐私计算赋能数据安全，图像隐私计算具备原始数据不出域、计算过程加密、不共享明文数据等优势，有效保障数据安全和用户隐私；打通数据孤岛，基于合法合规的数据利用，让原本不敢被共享的数据得以流通，打破数据壁垒，赋能多方协作、驱动业务创新；明确数据权属，通过分离数据所有权与使用权，不交易数据本身，只交易数据的计算结果，让数据交易与价值核算合理化。在图像隐私保护计算框架下，参与方的数据不出本地，实现“数据可用不可见、数据不动价值动”。

1.2 相关工作

随着网络技术的飞速发展和网络空间的不断延伸，数据呈现爆炸式增长，用户对于以人脸信息为代表的图像隐私数据的私密性要求也越来越高。而图像数据的不共享，不利于图像处理技术乃至整个图像应用前景的发展。用于图像处理的人工智能需要不断获取新的数据、进行持续且深度的学习。如何去实现大数据的价值，让用户更放心地交付隐私数据，这需从数据隐私保护、多方安全计算两个方面来进行加强。

数据隐私保护技术发展至今，较为成熟的是同态加密技术，被应用在众多隐私计算系统中。在这种加密模式下，对加密后数据进行处理得到的加密输出，在解密后，其结果与用同一方法处理未加密原始数据得到的输出结果是一样的。由于密钥仅在初始加密和最后解密阶段使用，整个计算处理过程中的输入和输出可以保持完全的隐私。目前存在许多基于同态加密的多方安全计算协议，如唐春明等人提出的基于多比特全同态加密的安全多方计算方案 [1][2]，基于 HE 的多方安全计算协议 [3][4][5]。但同态加密技术仍然有拖慢数据计算、精度损失、数据存储量大等缺点，不利于后期对海量数据集的训练。

此外，当前的多方安全计算大多进行于提取特征阶段，并没有训练模型阶段，只是简单地实现了加减乘除。在 Kai Huang 的模型 [6] 中，建立基于双服务器的卷积神经网络特征提取框架，充分利用 CNN 在移动传感器上有限的物理资源的优势，设计了一系列安全的交互协议，并利用两个边缘服务器协同执行 CNN 特征提取，但缺少训练过程的交互协议。

1.3 创新性

(1) 赋能数据安全：

本系统采用数据实例隐藏技术，对用户传入的图像数据进行加密存储。目前，用户图像数据主要是以明文传输的方式传入到系统，导致系统一旦被攻击，就会导致用户图像数据大量泄漏。针对该问题，本系统使用通过 MPC 改进后的 Instahide 算法对图像进行实例隐藏，通过综合运用 Mixup 数据增强的数据混合以及掩码处理等技术，在进行训练前对所有用户传入的原始数据进行简单、高效的加密，系统只存储加密后的图像，确保了在神经网络训练的过程中，即使数据被攻击者窃取、恢复，也无法还原出原始数据从而对图像数据进行了保护。

(2) 提供可靠算法：

本系统使用基于双服务器的分布式训练架构。目前常规的神经网络训练中，对受训练数据的隐私保护是一个普遍薄弱的环节。恶意攻击者不但能够通过最后的训练结果轻易推导出训练数据，甚至可以无需最后的结果，而仅通过在训练过程中传递的梯度参数就可以对他人的训练数据进行反向恢复，这对用户的数据安全是一个不小的潜在隐患。针对多方神经网络训练中的隐私保护问题，本项目设计了一种基于 InstaHide 实例隐藏方法的多方用户数据混合和加密技术。通过一种以 Mixup 数据混合方法为基础的 InstaHide 实例隐藏方法，在进行训练前将所有用户传入的数据进行混合并处理以达到对原始数据加密的效果，保证了即使在神经网络训练过程不能证明其安全性的前提下，恶意攻击者也无法仅通过攻击训练过程还原出用户上传的原始数据，提供当前分布式框架中的增强安全功能。同时，经过加密后的图像仍然具有训练能力，本系统基于 MPC 实现了对加密数据的训练算法，并取得了优良的性能。相对于同态加密，Instahide 实例隐藏方法可以达到速度更快、存储量更少、精度损失小，这有助于多源的大数据处理，也适应深度学习发展对于数据的要求。

(3) 打通数据孤岛：

本系统提出基于双服务器的神经网络训练协议，数据所有者可将他们的私有数据分布在两个非协作服务器合作进行特征提取以及模型训练。我们利用秘密共享技术完成两个服务器的交互，同时保证神经网络的准确性和数据的隐私安全性。本系统提供的可靠的数据协作训练系统，提高了共享数据隐私保护的安全性、在保护所有者数据隐私的情况下，大大提升了不同来源数据集共享的可能，打通了数据孤岛，让原本不敢被共

享的数据得以流通，打破数据壁垒，赋能多方协作、驱动业务创新。

(4) 分割数据权属：

现有的图像保护主要是依靠出台政策来规范管理数据，但该方法存在诸多弊端，如企业遵守程度、政府监管力度，这些都属于是人为因素，无法保障，因此即使制定了如此多的相关政策，图像隐私泄露问题仍然是一个难以整治的问题。而本系统从图像的角度出发，直接对图像进行保护处理，实现数据权属的明确。系统只具有图像的使用权，而不具有所有权，只有所属用户拥有图像的所有权。通过是分离数据所有权与使用权，不交易数据本身，只交易数据的计算结果，让数据交易与价值核算合理化。在保证图像数据应用价值的同时，避免了企业滥用数据，企业中存储图像被窃取等隐私泄露风险。

1.4 应用背景

随着计算机技术的不断发展，数字图像处理技术的应用也愈发广阔。数字图像处理技术的快速发展，使得图像数据具有更大的应用价值，被广泛运用于各个领域，包括医疗保健、资源产业、通信移动、航空航天、军事安全等方面。但图像数据已经成为了一种私有财产，大众对于数据的隐私性也越发敏感。与此同时，无论是医学方面的 X 图像增晰应用，军事方面的侦察图片分析应用，还是电子商务方面的人脸鉴别、指纹识别应用，所用的图像数据都有很高的隐私保障需求，也面临着很高的隐私安全威胁。本系统可以赋能数据安全，所采用的隐私计算方法具备原始数据不出域、计算过程加密、不共享明文数据等优势，有效保障数据安全和用户隐私。并且在保障图像数据不丧失应用价值的同时，实现所用图像数据的隐私保护，弥补了当前数字图像处理技术在安全性和隐私性上的不足。

针对当今数据因为私密性互相孤立、神经网络输入源稀缺的挑战，本项目将数据隐藏技术 Instahide[7] 和安全多方计算结合，设计了易操作、准确性高的双服务器训练系统。该双服务器架构可以实现双方合作训练，同时数据隐藏技术保证图像数据训练前后对于非所有方都是不可知的，也为抵御神经网络的梯度攻击提供了一定的保护，即使被攻击方窃取，攻击方也无法进行使用，大大增加了数据的隐私性和安全性，从而打通数据孤岛，基于合法合规的数据利用，让原本不敢被共享的数据得以流通，打破数据壁垒，赋能多方协作。本系统适用于任何图像隐私数据合作的应用背景，比如医疗行业中的疾病预测、金融行业的、舆情分析以及科研中的数据合作等，根据 Zhao 等人的研究

和实验 [8][9][10]，安全多方计算在解决这些领域的安全和隐私问题上有着巨大的优势，凡是需要多方隐私数据共同训练的场景，本系统都能够为其提供合作机会。

1.5 综合价值

商业价值方面，本系统具有广阔的市场规模。当下，金融机构、国家单位等使用的身份认证系统以人工智能技术为依托，以身份特征训练图像为训练性能保障，随着部分黑灰产业利用泄露的生物图像数据对系统进行攻击的问题日趋严峻，安全风险不断放大，大众对于隐私安全的关注度日益增加，根据 2022 年两会期间的市值预测，三年后隐私计算技术系统服务营收有望触达 100 亿 - 200 亿人民币的空间，甚至有望撬动千亿级的数据系统运营收入空间。我们的系统的使用隐私计算技术防止数据泄露，提高用户对企业的信任度，从而提升企业的竞争力与社会公信力，提高企业的商业价值。

发展潜力方面，在时间维度上，目前隐私计算市场仍处于大规模商业应用的前期，随着技术不断成熟和市场认知的提高，应用隐私计算的系统将持续为政府、金融、公共服务、工业互联网等高价值场景提供可信数据基础，为数字经济形态的向好发展提供稳健助力。在领域角度上，本系统具有向各类要求保护图像隐私的训练场景发展的内在动力。无论是身份认证识别系统，还是医疗图像训练，都有共同的隐私保护需求、基本一致的图像分类网络结构、以及相似的痛点问题，如训练过程中存在的数据隐私泄露问题，这些都可以通过本系统在一定程度上解决，因而本系统具有较好的业务扩展潜力。

落地可行性方面，首先，本系统支持高准确率训练，持续丰富业内各类高风险攻击样本数据集全面覆盖，新型安全风险，高效抵御恶意攻击。同时，本系统运算效率高，具有毫秒级图片检测速率，支持根据业务需求横向扩展检测能力，能够保证企业在使用多方安全计算技术的同时以高速率训练。此外，该产品部署方便，支持多类型文件检测，SDK 形态部署快捷，一体机形态开箱即用。

第二章 作品介绍

本系统的设计目标为在分布式双服务器的环境中实现图像的安全存储与安全训练。用户在上传图像数据时，会将已经秘密分享为两份的图像数据分别发往分布式的双服务器，通过参数传递将原本单服务器的训练任务分配到两台服务器上，在保证神经网络训练结果准确性的同时，克服了单服务器安全性脆弱的缺陷，提高了服务器训练过程的安全性和隐私保护性。当经过秘密分享的图像数据上传到两个服务器之后，数据会以 InstaHide 实例隐藏的方法进行数据混合和加密，保证了在神经网络的训练中，即使训练结果或梯度参数被恶意攻击、窃取，也无法还原得到原始数据，显著提升了该系统的抗攻击的安全性能。同时在神经网络训练过程中，建立了基于双服务器的 CNN 训练协议，进一步保证了系统的隐私性和安全性。

2.1 系统框架

本系统涉及四个实体角色，分别为用户群体、用户本地平台、日志服务器和双服务器。双服务器的作用是进行数据隐藏加密并训练；日志服务器的作用是存储日志信息，记录服务器计算过程和用户的项目操作行为；用户本地平台的作用是提供项目管理以及用户管理；用户群体拥有数据的管理权限和最终模型的拥有权限。

系统的框架如图1所示。用户可通过系统提供的离线插件上传图像数据的相关信息，便于用户进行查看和分享。数据不会在系统进行汇总，而是直接通过系统提供的接口以秘密份额的形式发布到两台服务器上，避免了用户图像数据的泄露。服务器进行准备工作，将之后需要使用的参数生成完全后，才开始数据隐藏与数据训练。其中数据隐藏与数据训练均基于系统的安全协议进行，保护用户数据的隐私性和安全性。

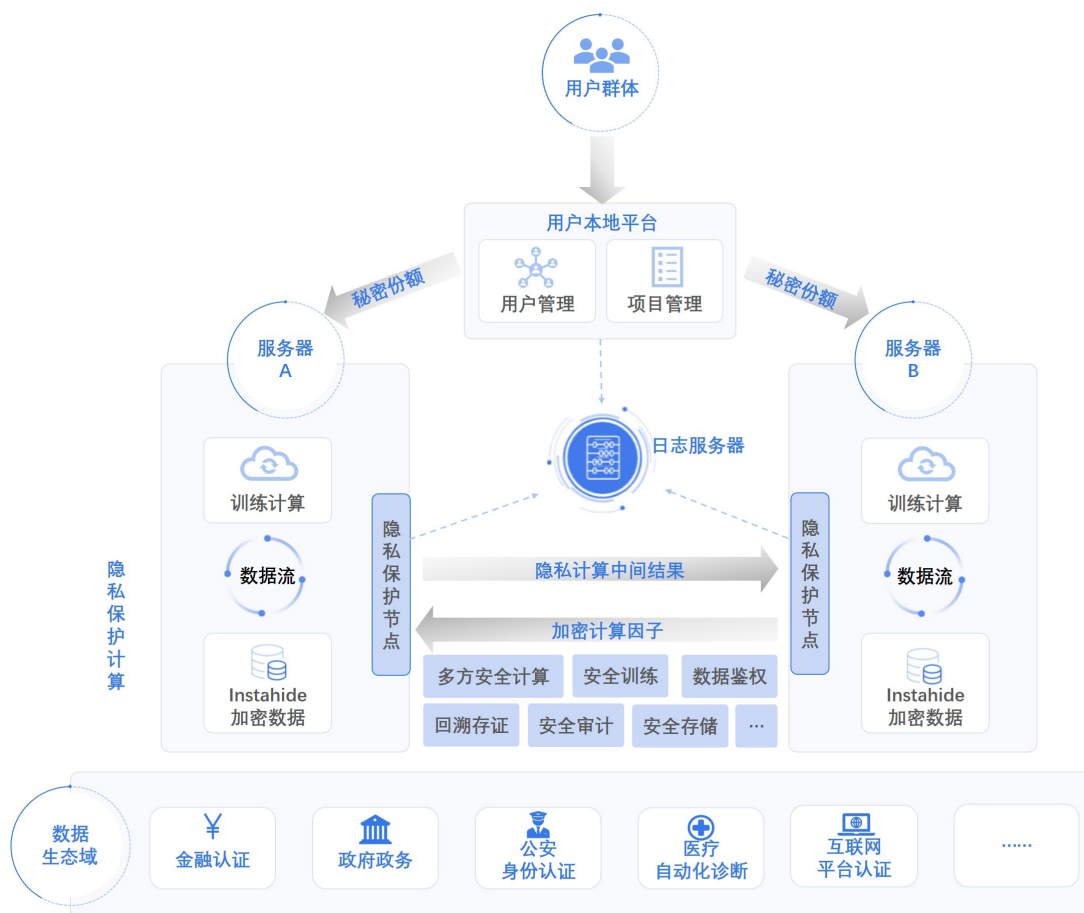


图 1: 系统框架图

系统的体系结构如图2所示。系统体系主要分为四层：具体应用层、对外接口层、系统功能层、基础设施层。每一层有着不同的功能模块，且下层对上层提供支撑。



图 2: 系统体系图

2.2 系统模块

本系统软件功能分为九个模块，分别是用户管理模块、数据管理模块、项目管理模块、系统日志模块、秘密共享模块、数据隐藏模块、参数生成模块、数据交换模块和神经网络训练模块。其中用户管理模块是对登录用户的身份管理和权限验证；数据管理模块在可视化的基础上支持数据的上传、存储操作；项目管理模块包括项目的可视化发起以及日志记录操作，主要判断用户角色的行为是否合法；系统日志模块采用 IPFS 对日志进行存储；秘密共享模块主要负责将各个用户的原始数据通过分享算法分为两份，以支持后续双服务器模型的数据隐藏和训练；数据隐藏模块主要负责将得到的数据份额进行加密隐藏，以加强训练数据自身的安全性，使原始数据难以被还原；参数生成模块主要包括三元组的生成和相关不经意传输（COT）的应用，为双服务器模型预先提供参数；数据交换模块主要负责在数据预处理以及数据训练过程中实现参数的交换和安全传输；神经网络训练模块采用神经网络进行数据隐藏后的特征提取以及模型训练。各模块及其相互关系图如图3所示。

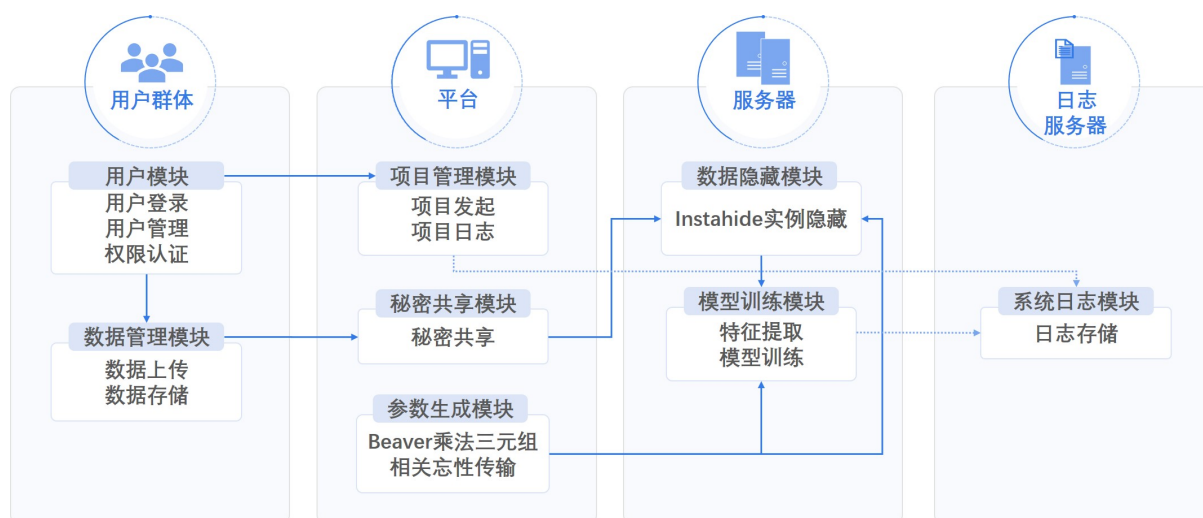


图3: 模块关系图

2.2.1 用户管理模块

用户管理模块是针对训练系统来说的，训练系统的作用是进行数据训练，其用户分为两种，分别是管理员和普通用户，普通用户又分为发起者与参与者。对于管理员用户，其可以执行查看系统日志、监控系统流量等操作，而普通用户仅仅具有创建、删除、执行、共享项目等权限，不同角色的用户对项目的权限也有所不同，这就涉及到不同用户、不同角色的权限管理。

2.2.2 数据管理模块

数据管理模块包含数据上传和数据管理两部分。

数据上传可由任意用户进行操作，用户可以借助系统提供的开源的离线插件处理本地数据，将数据集的相关信息上传到系统，系统会对相关信息如数据格式，数据大小，数据标签等进行检查，确认无误后系统记录该数据集，将该数据集的所有人置为该用户。当用户要将上传的数据用于训练时，系统会提供接口供用户将数据直接提供给训练服务器。

数据管理仅提供给数据集的所有者，用户可以定制上传到系统的数据集，设置该数据集是否可以使用，有多少条数据可以使用，哪些数据标签列是生效的等等。

2.2.3 项目管理模块

项目管理模块包含项目创建、项目共享、项目执行、项目展示以及日志记录。

项目创建可由普通用户进行操作，向系统服务器进行项目创建请求，系统服务器确认用户权限后，向用户返回一张表单供用户填写项目的具体信息以及为后续的项目共享、数据上传等操作提供基础。

项目共享可由项目创建者及已被邀请加入项目的普通用户进行操作，向系统服务器进行项目共享请求，系统服务器确认邀请用户权限及被邀请用户权限后，向被邀请用户发送该项目的基本信息及一个确认是否加入的表单。若被邀请用户向服务器确认加入，则服务器继续返回一张表单供用户项目邀请、上传数据。

项目执行只能由发起该项目的用户进行操作，在其确认所有数据均已上传成功后，向系统服务器进行项目执行请求，系统服务器确认该用户权限后，向训练服务器 S_0 、 S_1 发送训练请求，得到应答后向用户返回执行成功信息。

项目展示可由该项目的所有参与者进行操作，向系统服务器进行项目展示请求，系统服务器确认当前项目进度及用户权限，若项目已完成，则服务器返回训练结果给用户，若未完成，则返回当前训练进度。

日志记录由系统服务器进行操作，在项目未开始前记录用户的发起、邀请、加入、拒绝等一系列操作及操作时间，在项目开始后实时记录项目进度，项目完成后记录项目完成情况以及用户对项目结果的查看情况及反馈。该项目的所有参与者都可对该日志进行实时查看。

2.2.4 系统日志模块

本系统的日志模块主要是记录训练项目基本信息及操作状态，即训练项目的发起者、参与者，项目持续时间，训练过程是否成功等信息。日志的内容通过 IPFS 节点进行存储。

IPFS 又称星际文件系统，是一种内容可寻址的对等超媒体分发协议。在 IPFS 网络中的节点将构成一个分布式文件系统，任何存储在系统里的资源，包括文字、图片、声音、视频，以及网站代码，通过 IPFS 进行哈希运算后，都会生成唯一的地址。只要通过这个地址就可以打开文件进行查看。除此之外，由于加密算法的保护，该地址具备了不可篡改和删除的特性，一旦日志存储在 IPFS 中，它就会是永久性的，这也提高了日志系统的安全性。存储站点的分布式网络越多，它的可靠性也就越强。

2.2.5 秘密共享模块

秘密共享模块完成了用户在不透露完整数据的情况下向双服务器提供足以支撑训练的数据的过程，作用是使用户的数据完全私有，系统、其他用户以及服务器都无法获得。

秘密共享的示意图如图4所示。秘密分享的基本思想是将数据切割成多份，并分发给不同的参与者，每个参与者持有其中一份，协作完成计算任务（比如加法、乘法运算）。因为参与者看不到数据全量信息，从而实现数据隐私保护。

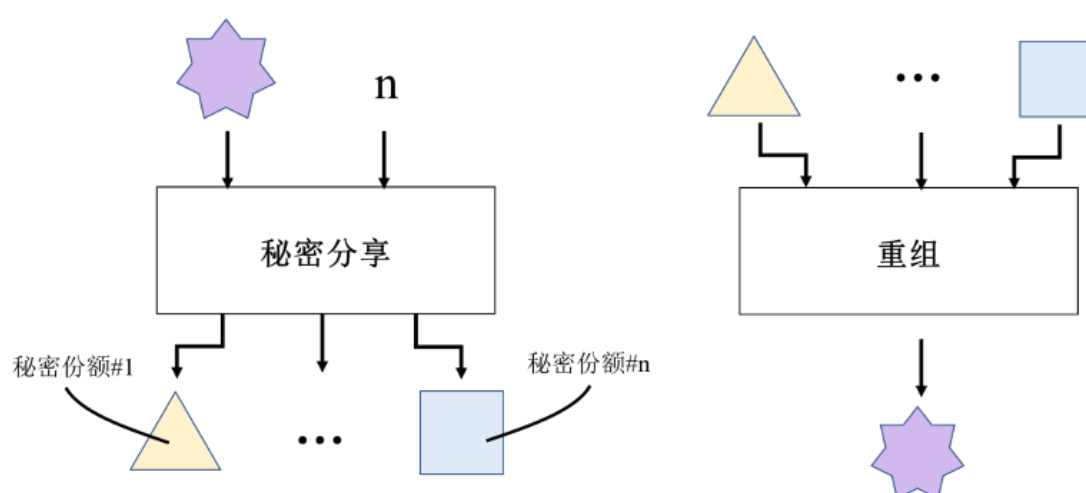


图 4: 秘密分享示意图

用户通过秘密共享的方式将数据分布在双服务器，可保证服务器在不获知完整原始数据的情况下，仍然能够进行训练。秘密共享的实现过程如下：对于数据集 D ，用户

通过分享算法将其分为两份，发给双服务器；训练结束后，服务器返回模型份额，用户拿到所有份额后合成完整模型。

服务器实际上拿到的是数据的秘密份额。秘密份额即原始数据的一部分，双服务器所持有的数据份额可以合成为一个完整的原始数据。为保证用户数据隐私性，双服务器必须处于非共谋状态，即使一方服务器被攻击，攻击者也无法得到用户的完整数据。而对于单方服务器来说，仅持有的数据份额虽然是原始数据的一部分，但没有任何意义，无法恢复出完整数据，也就无法为己所用。训练后返回给用户的结果也是模型份额，即模型的一部分，最终的完整模型由用户合成。自始至终，服务器都无法拿到完整的原数据及训练模型，保证了用户的数据隐私性和数据安全性。

秘密共享是现代密码学领域的一个重要分支，是信息安全和数据保密中的重要手段，也是多方安全计算和联邦学习等领域的一个基础应用技术。

2.2.6 数据隐藏模块

数据隐藏模块涉及到一个重要的算法，InstaHide 算法。

InstaHide 算法是一种基于数据增强算法 Mixup 的对神经网络训练原始数据进行隐藏的加密算法。Mixup 算法通过对私有数据集和公共数据集的多个数据进行混合以生成新的训练数据，进而达到增强训练集的效果。而 InstaHide 算法充分利用了 Mixup 算法混合数据而不破坏数据训练效果的优势，在此基础上通过添加掩码等方法进一步加强了混合后数据的隐秘性。

在本系统中，数据隐藏模块的作用在于保证了训练结果准确性的同时，也保证了训练数据的不可还原性。服务器在开始训练之前，会对收集到的数据进行 InstaHide 加密，这样即使有攻击者对服务器进行攻击并还原出了使用数据，那么其得到的也是 InstaHide 加密后的数据，而非用户的原始数据，增加了安全性。

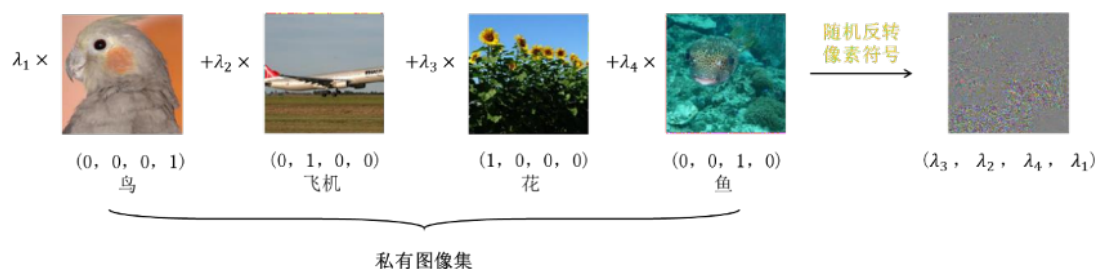


图 5: Instahide 加密效果图

Instahide 加密效果如图5所示。Instahide 应用到最左边的私有图像，从训练集中随

机选择三个图像进行混合，得到合成图像，接着在合成图像上进行符号反转，完成一张图的加密。加密每张图的比例因子 λ_i 是随机选择的。

同时，本系统的数据隐藏模块在常规 InstaHide 算法的基础上，提出了基于双服务器模型的 Instahide 多用户数据混合协议，可以将多个用户的数据以秘密分享的方式传输给双服务器，服务器以持有数据份额的形式进行 Instahide 加密，并在加密的过程中将必要的特定参数进行交互传递，在不泄露各方原始数据的基础上实现了双方服务器的合作加密。

2.2.7 参数生成模块

参数生成模块是指在进行数据预处理之前，将处理过程中大量需要的中间参数提前生成。在所涉及的计算中，用户的数据均通过乘法三元组的辅助来获得数据之间的乘积，相关不经意传输技术（COT）辅助三元组的产生。其中，乘法三元组即为所需要提前生成的中间参数。

乘法三元组指生成的随机数 a 、 b 及其乘积 $c = a \times b$ ，在无法得知完整乘数的情况下，利用乘法三元组既可以盲化原始数据，同时又能得到正确的乘积结果，解决了秘密分享的情况下如何计算乘积的问题。被大量运用到多方安全计算协议中。乘法三元组是全局不可见，全局不可见指任何一方都无法知道完整的 a 、 b 、 c ，它以份额的形式存在。在数据预处理之前，乘法三元组以份额的形式由服务器各自产生；在服务器进行运算时，乘法三元组也以份额的形式被使用。但 c 的计算需要知道完整的，所以用一种特殊的技术——COT 来得到 c 的结果。

乘法三元组是消耗性的，每次乘法计算都会消耗一个乘法三元组，通过预先计算的乘法三元组，将通信量和计算量移到了数据开始处理之前。

不经意传输（Oblivious Transfer，简称 OT）是一种密码学思想，其核心是发送方 S 提供所有消息且接收方 R 给出选择位的情况下，保证接收方 R 只能获取其中一个消息，而发送方 S 无法知道 R 获取的是哪一条消息。COT（Correlated Oblivious Transfer）是在 OT 上结合乘法三元组的实际情况进行的改动。COT 确保双方在不可知完整 a 、 b 的情况下，计算出各自应持有的 c 的份额。双方服务器拿到的只是乘法三元组随机数 a 、 b 、 c 的份额，完整的 a 、 b 是全局不可见的，各方服务器仍可通过 COT 计算 a 、 b 的乘积 c 并进行秘密分享。

2.2.8 数据交换模块

在数据预处理以及数据训练过程中,双服务器需要交换参数,本项目基于 SSL(Secure Sockets Layer) 协议保证数据的安全传输。

SSL 协议可分为两层: SSL 记录协议 (SSL Record Protocol): 它建立在可靠的传输协议 (如 TCP) 之上, 为高层协议提供数据封装、压缩、加密等基本功能的支持。SSL 握手协议 (SSL Handshake Protocol): 它建立在 SSL 记录协议之上, 用于在实际的数据传输开始前, 通讯双方进行身份认证、协商加密算法、交换加密密钥等。SSL 协议为服务器之间提供认证, 确保数据发送到正确的对象; 并对数据进行加密, 以防止数据中途被窃取或被改变。

2.2.9 神经网络训练模块

神经网络训练模块负责训练实现各种图像应用的神经网络, 即完成了数据隐藏后的特征提取以及模型训练。这里以最典型的卷积神经网络 CNN (Convolutional Neural Network, CNN) 为示例对象。

本质上, 卷积神经网络是一种带有卷积结构的多层前馈神经网络, 但区别于传统的全连接前馈神经网络, CNN 具有局部连接和参数共享的重要特征, 从而减少了连接和权值的数量, 降低了网络模型的复杂度, 提高了计算效率, 特别是网络规模越大、效果越显著。另外, CNN 通过层叠的卷积和下采样操作自动提取具有平移不变性的局部特征。一个完整的卷积神经网络可包含卷积层、池化层、全连接层等。其中卷积层用来进行特征提取, 池化层用于降低维数, 全连接层可用于结果预测。CNN 的关键点之一是前向传播提取特征, 输出的结果是每幅图像的特定特征空间。另一个关键点就是如何通过训练数据迭代调整网络权重, 也就是后向传播算法。通过后向传播算法, 不断调整模型中的权重和偏差直到最优解。CNN 可以自动从大规模数据中学习特征, 并把结果向同类型未知数据泛化, 实现数据的分类等。Mrazova 等人 [11] 还提出一种增长式 CNN, 实现快速自动调整网络拓扑结构、有效处理高维数据、逐层迭代提取高级抽象特征。Kaiming He 等人 [12] 则提出一种空间金字塔池化 CNN 算法, 实现不同尺寸图像的识别。CNN 目前在很多很多研究领域取得了巨大的成功, 并被应用于计算机视觉、自然语言处理等领域。

本项目建立了基于双服务器的 CNN 训练协议, 将 CNN 分布在两个服务器上, 通过安全的秘密共享技术进行参数交互, 完成提取特征以及后向传播学习过程。

其中输入是经过数据隐藏模块加密后的数据份额，输出为模型份额，最终在用户处进行合并，服务器在运算的过程中无法拿到有用数据，保证了计算过程中的图像数据的隐私性和安全性。

第三章 实现方案

3.1 开发环境

本节主要介绍本系统开发的软硬件配置信息和系统开发用到的工具。

3.1.1 设备配置

本系统的开发主要包括用户系统的搭建，用户数据的隐藏，分布式计算中关键参数的分发，分布式计算中关键数据的交换以及分布式神经网络的训练。实体所对应的硬件设备配置信息如表1所示，软件设备配置信息如表2所示。

表 1: 硬件设备配置信息表

设备名	硬件信息
用户主机 1	8g 内存 Intel(R) Core(TM) i5-8250U CPU
用户主机 2	12g 内存 Intel(R) Core(TM) i7-7500U CPU
用户主机 3	8g 内存 Intel(R) Core(TM) i7-7700HQ CPU
训练服务器 1	196g 内存 Intel(R) Xeon(R) Gold 6248 CPU
训练服务器 2	196g 内存 Intel(R) Xeon(R) Gold 6248 CPU
用户系统服务器	16g 内存 Intel(R) Core(TM) i5-7300HQ CPU

表 2: 软件设备配置信息表

设备名	软件信息
用户主机	Windows10 操作系统 Google Chrome 浏览器
训练服务器	Ubuntu 18.04.3 LT
用户系统服务器	Windows10 操作系统 Google Chrome 浏览器

3.1.2 开发工具

本系统的开发主要使用 VS Code，PyTorch 和 Vue.js，分别作为 IDE（集成开发环境），软件库和前端框架。VS Code 全称 Visual Studio Code，是微软于 2015 年推出的

一款免费开源的现代化轻量级代码编辑器，跨系统支持 Windows，Linux 和 MacOS。支持几乎所有主流的开发语言的语法高亮、智能代码补全、自定义热键、括号匹配、代码片段、代码对比 Diff、Git 等特性，支持插件扩展，并针对网页开发和云端应用开发做了优化。PyTorch 是使用 GPU 和 CPU 优化的深度学习张量库，基于 Torch，底层由 C++ 实现，其具有两大特征：类似于 Numpy 的张量计算，可以使用 GPU 加速；基

于带自动微分系统的深度神经网络。这两大特征极大的提升了运算速度，降低了开发难度。Vue.js（简称为 Vue）是一个用于创建用户界面的开源 JavaScript 框架，在 MVC 模型（模型-视图-控制器）中 Vue 的核心只关注视图层，可以自底向上逐层应用。如果与现代化的工具链和各种支持类库结合使用时，Vue 也能创建复杂的单页应用。

3.2 Instahide 的实现方案

数据隐藏模块完成了对原始数据的预处理及隐藏操作，主要分为两部分：第一步，将原始数据进行秘密分享，分发给两个服务器；第二步，双服务器根据已生成的三元组等参数，执行 Instahide 协议，对原始数据进行隐藏。数据隐藏模块具体流程图如图6所示：

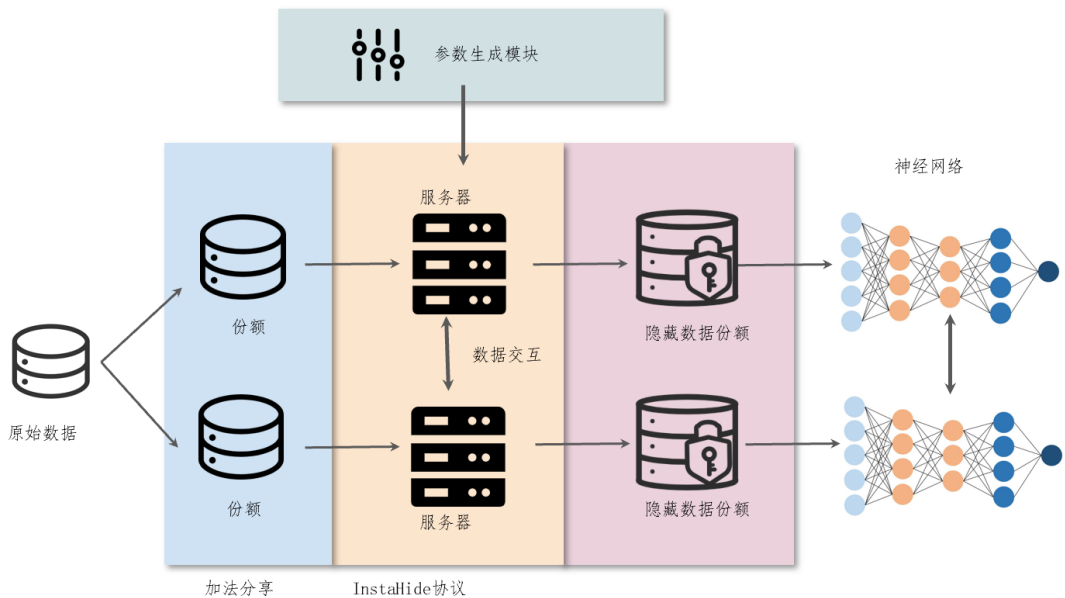


图 6: 数据隐藏模块流程图

3.2.1 秘密分享

在本系统中，秘密分享算法是数据隐藏模块基础算法，该算法能保证用户数据对单方服务器不可见。本系统主要使用了加法秘密分享，具体过程为引入与分享初始数据同维度的随机矩阵，通过在大数域 P 上的线性减法运算，将单个数据矩阵分享为两个随机矩阵，之后分别发送给两个服务器进行进一步运算，加法分享流程示意如图7所示：

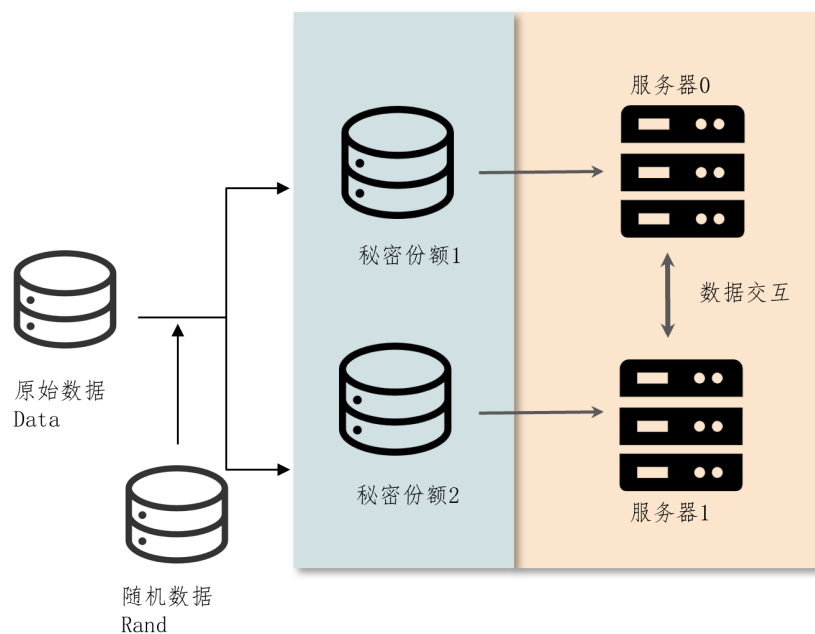


图 7: 加法分享示意图

随机数据 Rand 的引入可以保证秘密份额的随机性，其随机性由 Rand 决定。

当使用数据矩阵时，两方可以通过交换份额，运行重建算法，即可获取数据矩阵。
秘密份额重建流程如图8所示：

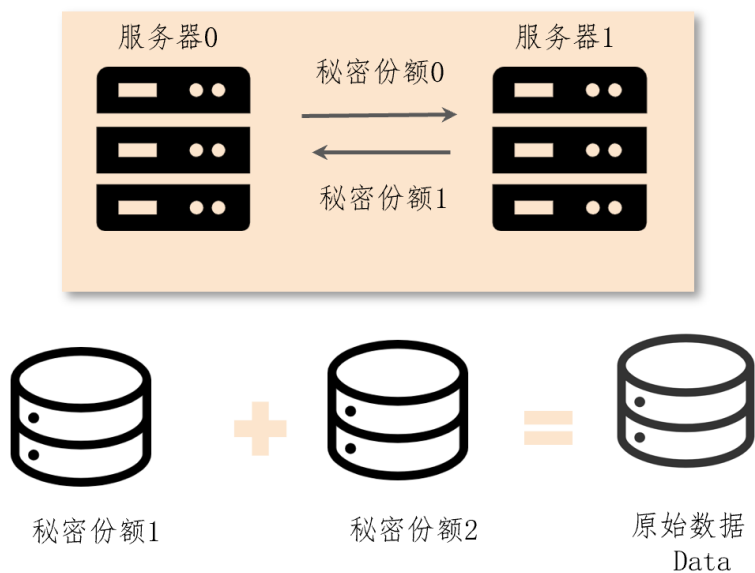


图 8: 加法分享重建示意图

在本系统中，原始数据在被用户分享并发送至服务器后，将一直保持秘密份额状态，在所有运算结束后，服务器会将模型的秘密份额发送至用户系统，由用户系统运行

重建算法后，返回完整的模型给用户。

3.2.2 双方 Instahide 协议

双方 Instahide 协议基于双服务器模型，能实现多个数据集的安全混合。该协议主要基于 Beaver 三元组实现。下图为 Instahide 协议的主要流程，详细过程如算法1所示：

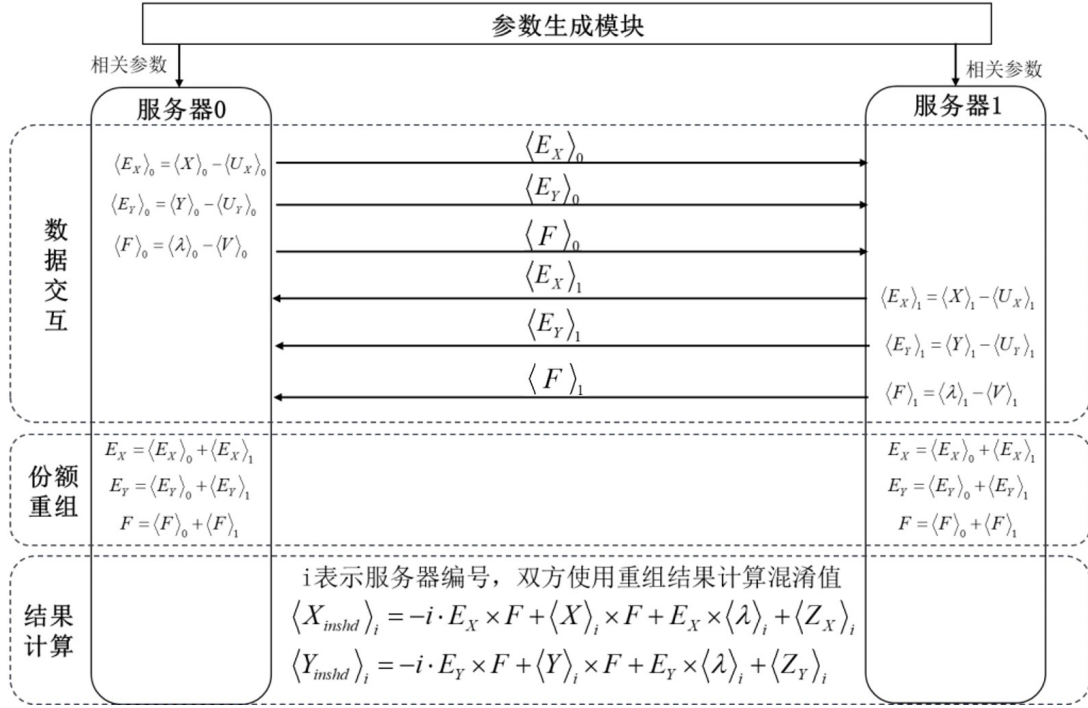


图 9: Instahide 流程

如图9所示，双方服务器获得参数生成模块生成的参数后，开始执行双方 Instahide 协议。在交互阶段，服务器首先使用随机矩阵隐藏自己的数据份额和比例份额，之后将隐藏后的份额（用 E_X , E_Y , F 表示）发送给对方；在计算阶段，双方首先对份额进行重建，之后通过 Beaver 三元组乘法来计算混合结果，从而拿到混合后的数据份额。

Algorithm 1 Instahide 协议

输入： 数据份额 $\langle X \rangle$ 和 $\langle Y \rangle$ ；比例因子份额 $\langle X \rangle$ 。其中 $\langle \rangle$ 用以表示秘密份额

两组 Beaver 三元组 $(\langle U_X \rangle, \langle V \rangle, \langle Z_X \rangle)$ ，用于隐藏数据和比例因子 $\langle \lambda \rangle$

数据选择参数 π ，用以选择 $k = 4$ 张图，将其混合为一张图片

输出： 经过 Instahide 算法充分混合的数据份额 $\langle X_{inshd} \rangle$ 和 $\langle Y_{inshd} \rangle$ ，用于下一步训练

- 1: $\langle E_X \rangle_i = \langle X \rangle_i - \langle U_X \rangle_i$ ，其中 $i \in \{0, 1\}$ 为服务器编号，将 $\langle E_X \rangle_i$ 发送给对方服务器
- 2: 双方计算 $E_X = \text{Rec}(\langle E_X \rangle_0, \langle E_X \rangle_1) = \langle E_X \rangle_0 + \langle E_X \rangle_1$
- 3: $\langle E_Y \rangle_i = \langle Y \rangle_i - \langle U_Y \rangle_i$ ，将 $\langle E_Y \rangle_i$ 发送给对方服务器

- 4: 双方计算 $E_Y = \text{Rec}(\langle E_Y \rangle_0, \langle E_Y \rangle_1) = \langle E_Y \rangle_0 + \langle E_Y \rangle_1$
- 5: **for** t from 1 to $epoch$ **do**
- 6: $\langle F \rangle_i = \langle \lambda \rangle_{i,t} - \langle V \rangle_i$, 将 $\langle F \rangle_i$ 发送给对方服务器
- 7: 双方计算 $F = \text{Rec}(\langle F \rangle_0, \langle F \rangle_1) = \langle F \rangle_0 + \langle F \rangle_1$
- 8: 双方计算 $\langle X_{mix} \rangle_i = \sum_{k=1}^4 \left(\langle X \rangle_{i,\pi_k} \times F + \langle \lambda \rangle_{i,k} \times E_{X,\pi_k} - i \cdot E_{X,\pi_k} \times F + \langle Z_X \rangle_{i,t} \right)$
- 9: 双方计算 $\text{Sign}_X = \text{Hash}_X(F)$
- 10: 双方计算 $\langle X_{inshd} \rangle_i = (-1)^{\text{Sign}_X} \times \langle X_{mix} \rangle_i$
- 11: 双方计算 $\langle Y_{mix} \rangle_i = \sum_{k=1}^4 \left(\langle Y \rangle_{i,\pi_k} \times F + \langle \lambda \rangle_{i,k} \times E_{Y,\pi_k} - i \cdot E_{Y,\pi_k} \times F + \langle Z_Y \rangle_{i,t} \right)$
- 12: 双方计算 $\text{Sign}_Y = \text{Hash}_Y(F)$
- 13: 双方计算 $\langle Y_{inshd} \rangle_i = (-1)^{\text{Sign}_Y} \times \langle Y_{mix} \rangle_i$
- 14: 使用 $\langle X_{inshd} \rangle$ 和 $\langle Y_{inshd} \rangle$ 进行训练
- 15: **end for**

Instahide 协议中所使用的关键参数来自于参数生成模块，其中运算的正确性由 Beaver 三元组保证。为了提高运算效率，本系统在实现时大量使用了矩阵运算，能高效地完成大数据集的处理。在 Instahide 协议执行结束后，服务器将继续基于得到的混合数据份额，进行进一步的模型训练。

3.3 参数生成模块

参数生成模块主要负责生成 Instahide 协议使用的参数及三元组，其主要流程如图 10 所示：

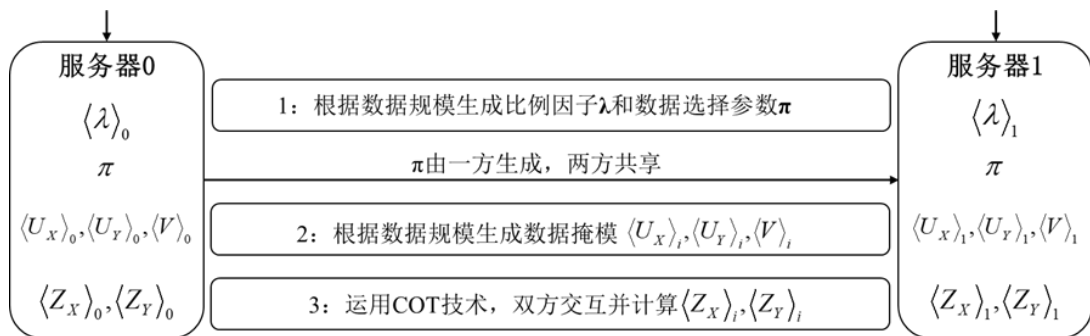


图 10: 参数生成模块

3.3.1 Beaver 乘法三元组

Beaver 预计算乘法三元组主要用于多方安全计算协议中的乘法计算，应用范围为加法和乘法均为线性的秘密分享机制，它的通信和计算应在协议开始前完成，协议在计算时可以直接使用三元组进行计算。在双服务器模型下，我们假设有两方计算的参与者 P_0, P_1 他们分别持有 a, b 两个数的加法分享秘密份额 $\langle a \rangle_i, \langle b \rangle_i$ 并希望计算 $a \cdot b$ ，且不对对方泄露自己持有的份额，使用 Beaver 预计算乘法三元组进行计算。

Beaver 预计算乘法三元组包括 $U, V, Z = U \cdot V$ 进行加法分享，分发给 P_0, P_1 ，即 P_i 持有 $\langle a \rangle_i, \langle b \rangle_i, \langle U \rangle_i, \langle V \rangle_i, \langle Z \rangle_i$ ，此时已经完成了三元组的准备，通过算法2进行计算：

Algorithm 2 Beaver 预计算乘法三元组的使用

- 1: 双方计算 $\langle E \rangle_i = \langle a \rangle_i - \langle U \rangle_i$ ，并将 $\langle E \rangle_i$ 发送给对方
 - 2: 双方运行重建算法 $E = \text{Rec}(\langle E \rangle_0, \langle E \rangle_1) = \langle E \rangle_0 + \langle E \rangle_1$
 - 3: 双方计算 $\langle F \rangle_i = \langle b \rangle_i - \langle V \rangle_i$ ，并将 $\langle F \rangle_i$ 发送给对方
 - 4: 双方运行重建算法 $F = \text{Rec}(\langle F \rangle_0, \langle F \rangle_1) = \langle F \rangle_0 + \langle F \rangle_1$
 - 5: 双方计算 $\langle a \cdot b \rangle_i = -i \cdot E \times F + \langle a \rangle_i \times F + E \times \langle b \rangle_i + \langle Z \rangle_i$
-

计算结果验证：

$$\begin{aligned}
 \text{Rec}(\langle a \cdot b \rangle_0, \langle a \cdot b \rangle_1) &= \langle a \rangle_0 \times F + E \times \langle b \rangle_0 + \langle Z \rangle_0 - E \times F + \langle a \rangle_1 \times F + E \times \langle b \rangle_1 + \langle Z \rangle_1 \\
 &= a \cdot F + E \cdot b + Z - E \times F \\
 &= a \cdot (b - V) + (a - U) \cdot b + U \times V - (a - U) \cdot (b - V) \\
 &= a \cdot b - a \cdot V + a \cdot b - U \cdot b + U \times V - a \cdot b + a \cdot V + U \cdot b + U \times V \\
 &= a \cdot b
 \end{aligned}$$

此后通过使用 Beaver 预计算乘法三元组，我们可以保证在份额状态下进行乘法运算，双方拿到乘法结果的份额。

3.3.2 参数生成模块主要运算

对于每个 epoch, Instahide 协议参数生成阶段生成随机的 π 和 λ 份额，我们将控制每方的比例因子和，使 $\sum_{k=1}^4 \langle \lambda_k \rangle_i = 0.5$ ，以保证在重建时获得正确的比例因子。

在三元组计算阶段，首先要根据 $\langle X \rangle, \langle Y \rangle, \langle \lambda \rangle$ 的维度，生成随机矩阵 $\langle U_X \rangle, \langle U_Y \rangle, \langle V \rangle$ ，随机矩阵将被用于 Instahide 协议中，对原始数据和比例因子份额进行隐藏。

完成 $\langle U \rangle, \langle V \rangle$ 的生成后, 双方服务器将通过 COT[13] (相关不经意传输) 或 LHE[14] (线性同态加密) 进行 $\langle Z \rangle$ 的计算, 保证 $Z = U \times V$, 且无须第三方来生成三元组并分发。在实现时, 我们使用了 COT 和 LHE 分别实现对 $\langle Z \rangle$ 的计算。因为:

$$\begin{aligned} Z &= U \times V = (\langle U \rangle_0 + \langle U \rangle_1) \times (\langle V \rangle_0 + \langle V \rangle_1) \\ &= \langle U \rangle_0 \times \langle V \rangle_0 + \langle U \rangle_0 \times \langle V \rangle_1 + \langle U \rangle_1 \times \langle V \rangle_0 + \langle U \rangle_1 \times \langle V \rangle_1 \end{aligned}$$

完成 Z 的分享本质上可以对上式中的四部分进行分享, 其中 $\langle U \rangle_i \times \langle V \rangle_i$ 可以由服务器单独计算完成, 对于剩余部分, 服务器通过 COT 或者 LHE 进行计算, 获得份额。我们令:

$$S_0 : \langle Z \rangle_0 = \langle U \rangle_0 \times \langle V \rangle_0 + \langle \langle U \rangle_0 \times \langle V \rangle_1 \rangle_0 + \langle \langle U \rangle_1 \times \langle V \rangle_0 \rangle_0$$

$$S_1 : \langle Z \rangle_1 = \langle U \rangle_1 \times \langle V \rangle_1 + \langle \langle U \rangle_0 \times \langle V \rangle_1 \rangle_1 + \langle \langle U \rangle_1 \times \langle V \rangle_0 \rangle_1$$

在 LHE 模式下, 以计算 $\langle U_0 \rangle \times \langle V_1 \rangle$ 为例, 计算流程如图11:

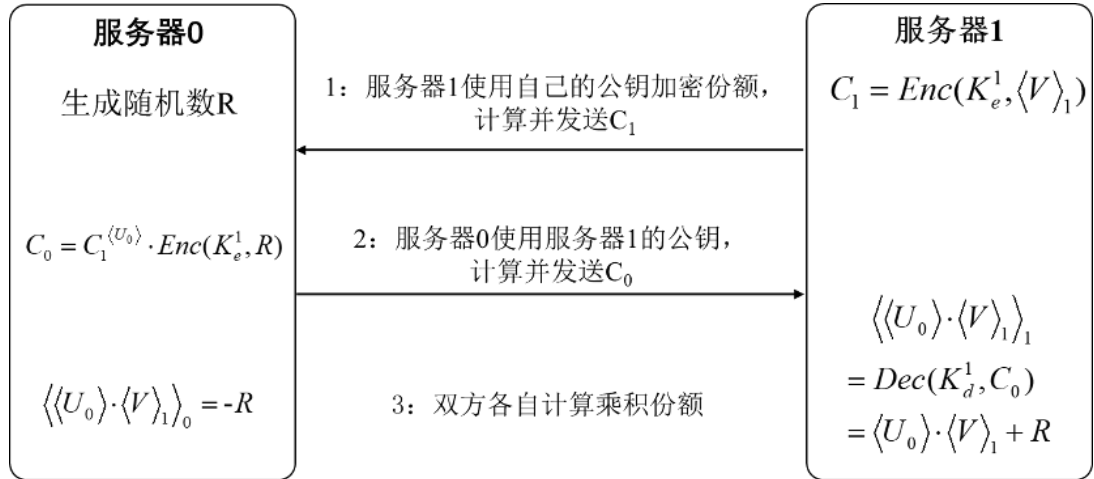


图 11: LHE 线性同态加密流程图

因 LHE 的同态性, 图中第 4 步 $\text{Dec}(c) = \text{Dec}(\text{Enc}(\langle V \rangle_1)^{\langle U \rangle_0} \cdot \text{Enc}(R)) = \langle U \rangle_0 \cdot \langle V \rangle_1 + R$, 从而实现了乘积的分享。

LHE 算法逻辑简单, 但因数据量大, 在使用时需要进行大量的公钥加密, 在测试时发现效率较低, 不适合于应用于系统, 故最终选择了 COT 模式, 下面进行 COT[13] 算法的介绍。

仍以 $\langle U \rangle_0 \times \langle V \rangle_1$ 为例, 完整的 COT 交互流程如图12所示:

在 COT 过程中使用了 OT[15] 对随机数种子进行传输, 在经过最后一轮交互后, 服务器即可单独计算 $\langle U \rangle_0 \times \langle V \rangle_1$ 的份额, 在系统实现过程中, 尽可能将所有运算放置于矩

阵中运算, 以提高效率。完整三元组计算过程如算法3所示:

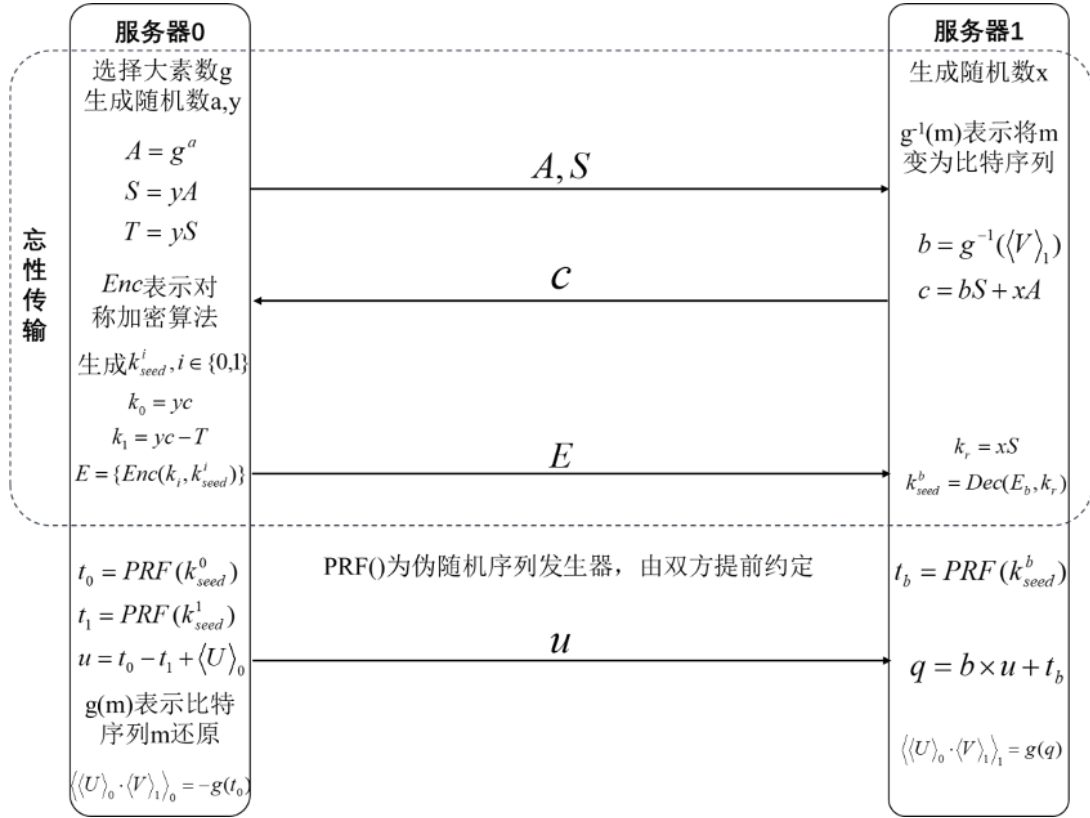


图 12: 计算三元组完整流程

Algorithm 3 COT

输入: 已生成的随机隐藏矩阵 $\langle U_X \rangle, \langle U_Y \rangle, \langle V \rangle$

数据选择参数 π

输出: 隐藏矩阵乘积份额 $\langle Z_X \rangle, \langle Z_Y \rangle$

- 1: 假设 $\langle U_X \rangle, \langle U_Y \rangle, \langle V \rangle$ 元素长度为 l 比特, 所有运算在大数域 $\mathbf{Z}_p, p$ 的原根为 g
- 2: 生成随机序列 $\mathbf{a}$, 计算 $\mathbf{A}_{\text{self}} = g^{\mathbf{a}} \bmod p$
- 3: 生成随机序列 $\mathbf{y}$, 计算 $\mathbf{S}_{\text{self}} = \mathbf{A}_{\text{self}} \times \mathbf{y}, \mathbf{T} = \mathbf{S}_{\text{self}} \times \mathbf{y}$
- 4: 将 $\mathbf{A}_{\text{self}}, \mathbf{S}_{\text{self}}$ 发送给对方服务器, 接收对方发送的 $\mathbf{A}_{\text{rec}}, \mathbf{S}_{\text{rec}}$
- 5: 计算 $\mathbf{b} = g^{-1}(\langle V \rangle)$, 其中函数 g^{-1} 表示将 $\langle V \rangle$ 转化为比特序列
- 6: 生成随机序列 $\mathbf{x}$, 计算 $\mathbf{c}_{\text{self}} = \mathbf{b} \times \mathbf{S}_{\text{rec}} + \mathbf{x} \times \mathbf{A}_{\text{rec}} \bmod p$
- 7: 计算 $\mathbf{k}_{\text{ot}, \text{rec}} = \mathbf{x} \times \mathbf{S}_{\text{rec}} \bmod p$
- 8: 将 $\mathbf{c}_{\text{self}}$ 发送给对方服务器, 接收对方服务器发送的 $\mathbf{c}_{\text{rec}}$
- 9: 生成随机数种子 $\mathbf{k}_{\text{seed}} = \{\mathbf{k}_{\text{seed}, 0}, \mathbf{k}_{\text{seed}, 1}\}$
- 10: 计算 $\mathbf{k}_{\text{ot}} = \{\mathbf{k}_{\text{ot}, 0} = \mathbf{y} \times \mathbf{c}_{\text{rec}} \bmod p, \mathbf{k}_{\text{ot}, 1} = \mathbf{y} \times \mathbf{c}_{\text{rec}} - \mathbf{T} \bmod p\}$
- 11: 计算 $\mathbf{E}_{\text{seed}, \text{self}} = \{\text{Enc}(\mathbf{k}_{\text{seed}, 0}, \mathbf{k}_{\text{ot}, 0}), \text{Enc}(\mathbf{k}_{\text{seed}, 1}, \mathbf{k}_{\text{ot}, 1})\}$

```

12: 将  $\mathbf{E}_{\text{seed,self}}$  发送给对方服务器, 接收对方服务器发送的  $\mathbf{E}_{\text{seed,rec}}$ 
13: 使用  $\mathbf{k}_{\text{ot,rec}}$  解密  $\mathbf{E}_{\text{seed,rec}}$ , 即计算  $\mathbf{k}_{\text{seed,rec}} = \text{Dec}(\mathbf{E}_{\text{seed,rec}}, \mathbf{k}_{\text{ot,rec}})$ 
14: for  $i$  from 1 to  $Epochs$  do
15:   生成随机序列,  $\mathbf{t}_{0,X} = PRF_X(\mathbf{k}_{\text{seed},0}), \mathbf{t}_{1,X} = PRF_X(\mathbf{k}_{\text{seed},1})$ , 其中  $PRF()$  表示
      随机数发生器
16:   计算  $\mathbf{u}_{X,\text{self}} = \mathbf{t}_{0,X} - \mathbf{t}_{1,X} + \sum_{k=1}^4 \langle U_X \rangle_{\pi_{m,k}}$ 
17:   将  $\mathbf{u}_{X,\text{self}}$  发送给对方服务器, 接收对方服务器发送的  $\mathbf{u}_{X,\text{rec}}$ 
18:   生成随机序列,  $\mathbf{t}_{b,X} = PRF_X(\mathbf{k}_{\text{seed,rec}})$ 
19:   计算  $\mathbf{q}_X = \mathbf{b} \times \mathbf{u}_{X,\text{rec}} + \mathbf{t}_{b,X}$ 
20:   计算  $\langle Z_X \rangle_m = \sum_{k=1}^4 \left( \langle U_X \rangle_{i,\pi_{m,k}} \times \langle V \rangle_i - g(\mathbf{t}_{0,X,k}) \right) + g(\mathbf{q}_X) \bmod 2^{2^l}$ , 其中  $g(\mathbf{x})$ 
      函数表示运算  $\mathbf{g} \times \mathbf{x}$ , 向量  $\mathbf{g} = (1, 2^1, \dots, 2^{l-1})$ 
21:   使用  $\langle U_Y \rangle$  替换  $\langle U_X \rangle$ , 重复上述 15-20 步骤, 计算  $\langle Z_Y \rangle_m$ 
22: end for

```

3.4 双服务器神经网络训练框架的实现方案

双服务器神经网络训练模块完成了图像数据隐藏后的图像特征提取以及模型训练, 即完成了多种图像应用中的神经网络的训练。这里以最典型的卷积神经网络 CNN(Convolutional Neural Network, CNN) 为示例对象。

CNN 是是一类包含卷积计算且具有深度结构的前馈神经网络, 它将输入层经过一系列隐藏层的前向学习得到输出层, 即特征向量空间。常用的隐藏层是卷积层 (Convolutional layer)、激活层 (activation layer)、池化层 (pooling layer)、全连接层 (fully connected layer)。接着进行后向传播, 通过计算误差, 从后往前调整隐藏层中的权重参数与偏差参数, 再次进行前向学习, 以此循环往复, 直到权重参数以及偏差参数达到最优解。

我们基于秘密分享技术设计了双服务器卷积神经网络安全计算框架, 双服务器拿到数据隐藏后的秘密份额, 并作为协议的输入完成项目。用户可以使用该框架直接利用现有模型进行分类, 或者进行新模型的生成训练。

3.4.1 双服务器神经网络模型

模型框架如图13所示，框架主要由三个主要实体组成：用户 U 、系统 T 和计算服务器 S_1 和 S_2 。服务器 S_1 、 S_2 以份额的持有原始图像经过秘密隐藏后的加密图像，将该秘密份额作为神经网络的输入。系统 T 负责计算过程中随机值的生成。特征提取以及训练过程由 S_1 、 S_2 完成。在通过以一些安全交互协议在密文域中运行 CNN 之后， S_1 和 S_2 将加密的 ϕ' 和 ϕ'' 返回给用户 U ，然后，用户 U 最终可以从他们的加密版本中恢复 CNN 特征 ϕ 。我们的框架基于半诚实模型，双方服务器不共谋，最少有一方服务器为诚信服务器，不会窃取数据发起攻击。

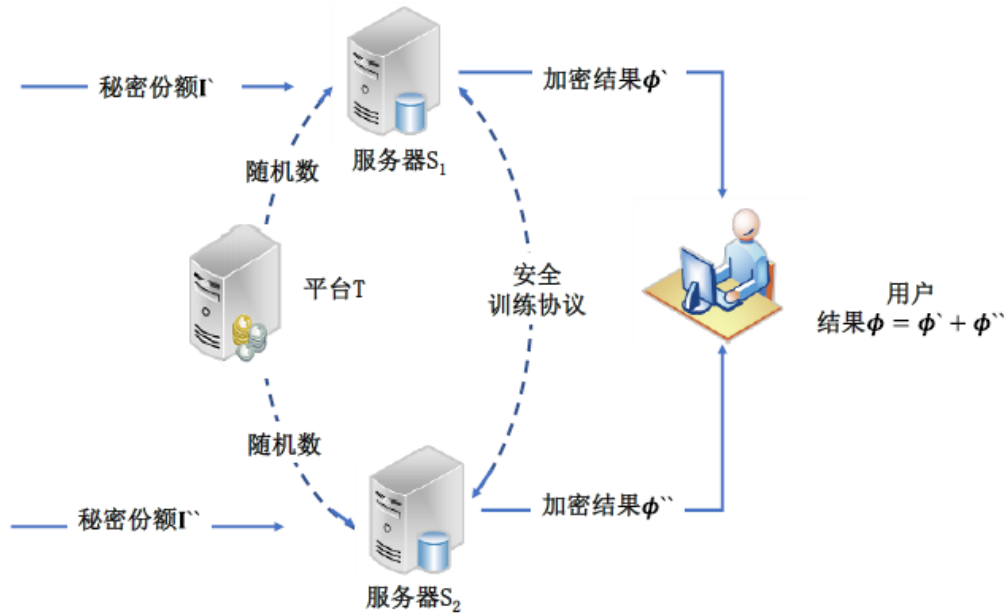


图 13: 双服务器神经网络训练模型

3.4.2 协助算法

在双服务模型中，我们利用以下三种算法 [6] 来协助完成安全计算。

I. 安全乘法算法 (SecMul)

该算法设计的目的在于配合 Beaver 乘法三元组，有效地减少计算和传输的代价，得到一个安全的乘法协议。该算法将隐秘生成的三元组 a 、 b 、 c 和 u 、 v 构成的线性组合 α 、 β 公开，进行运算，得到 u 、 v 的乘积 f ，而不会泄露任何有关 u 、 v 的信息。算法过程如算法4所示。

Algorithm 4 SecMul 算法

输入: S_1 提供输入 $u_1, v_1 \in F$, S_2 提供输入 $u_2, v_2 \in F$

输出: S_1 提供输出 f_1 , S_2 提供输出 f_2

离线状态:

- 1: 系统生成随机数 $a, b \in F$, 计算 $c = a \cdot b$
- 2: 系统将 a, b, c 分为随机份额: $a = a_1 + a_2, b = b_1 + b_2, c = c_1 + c_2$
- 3: 系统将 a_i, b_i, c_i 发送给 S_i

在线阶段:

- 4: 双方计算 $\alpha_i = u_i - a_i, \beta_i = v_i - b_i$, 并将 α_i, β_i 发送给对方
 - 5: S_1 计算 $\alpha = \alpha_1 + \alpha_2, \beta = \beta_1 + \beta_2$, 同时计算 $f_1 = c_1 + b_1 \cdot \alpha + a_1 \cdot \beta$
 - 6: S_2 计算 $\alpha = \alpha_1 + \alpha_2, \beta = \beta_1 + \beta_2$, 同时计算 $f_2 = c_2 + b_2 \cdot \alpha + a_2 \cdot \beta + \alpha \cdot \beta$
-

II. 安全位加法算法 (Bitadd)

为了尽可能的减少两台服务器之间的传输轮次, 我们基于 RCA 设计了安全加法协议 BitAdd, 我们从最低位往最高位迭代来依次计算每一位的和并将进位往前传导, 这样我们的计算中就会只包含异或运算和与运算。我们可以看到, 大量的异或运算可以在不交互的情况下进行本地运算, 而与运算则可以通过调用先前的安全乘法协议 SecMul 进行操作。Bitadd 的算法如算法5所示。

Algorithm 5 Bitadd 算法

输入: S_1 提供输入 $v_1^{(l-1)}LV_1^{(0)}$ 和 $r_1^{(l-1)}Lr_1^{(0)}$, S_2 提供输入 $v_2^{(l-1)}Lv_2^{(0)}$ 和 $r_2^{(l-1)}Lr_2^{(0)}$

输出: S_1 提供输出 $u_1^{(l-1)}Lu_1^{(0)}$, S_2 提供输出 $u_2^{(l-1)}Lu_2^{(0)}$

- 1: **for** j from 0 to $(l-1)$ **do**
 - 2: S_i 计算 $\alpha_i^{(j)} = V_i^{(j)} \oplus r_i^{(j)}$,
 - 3: S_1 和 S_2 共同计算 $(\beta_1^{(j)}, \beta_2^{(j)}) = \text{SecMul}(V_1^{(j)}, v_2^{(j)}, r_1^{(j)}, r_2^{(j)})$
 - 4: **end for**
 - 5: S_i 设 $c_i^{(0)} = 0$
 - 6: S_i 计算 $u_i^{(0)} = V_i^{(0)} \oplus r_i^{(0)}$
 - 7: **for** j from 1 to $(l-1)$ **do**
 - 8: S_1 和 S_2 共同计算 $(\alpha_1^{(j-1)}, \alpha_2^{(j-1)}) = \text{SecMul}(\alpha_1^{(j-1)}, \alpha_2^{(j-1)}, c_1^{(j-1)}, c_2^{(j-1)})$
 - 9: S_i 计算 $c_i^{(j)} = \alpha_i^{(j-1)} \oplus \beta_i^{(j-1)}$
 - 10: S_i 计算 $u_i^{(j)} = V_i^{(j)} \oplus r_i^{(j)} \oplus c_i^{(j)}$
 - 11: **end for**
 - 12: S_i **return** $u_i^{(l-1)}Lu_i^{(0)}$
-

III. 安全位提取算法 (BitExtra)

安全位提取算法基于 Bitadd 算法的基础上, 用于将共享数 u 的最高位提取, 以判断 u 的符号。在后续的运算中用于两个共享数的大小比较。BitExtra 算法如算法6所示。

Algorithm 6 BitExtra 算法

输入: S_1 提供输入 $u_1 \in \phi_{2^l}$, S_2 提供输入 $u_2 \in \epsilon_{2^l}$

输出: S_1 提供输出 $u_1^{(1-1)}$, S_2 提供输出 $u_2^{(1-1)}$

离线状态:

- 1: 对从 0 到 $(1-1)$ 的每一位 j , 系统生成随机数 $r_1^{(j)}, r_2^{(j)} \in \phi_2$
- 2: 对从 0 到 $(1-1)$ 的每一位 j , 系统计算 $r^{(j)} = r_1^{(j)} \oplus r_2^{(j)}$
- 3: 系统计算 $r = -r^{(l-1)} \cdot 2^{l-1} + \sum_{j=0}^{l-2} r^{(j)} \cdot 2^j$
- 4: 系统生成随机数 $S_1 \in \phi_{2^1}$, 同时计算 $S_2 = r - S_1$
- 5: 系统将 S_i 和 $r_i^{(1-1)} Lr_i^{(0)}$ 发送给 S_i

在线阶段:

- 6: S_1 计算 $t_1 = u_1 - S_1$
 - 7: S_2 计算 $t_2 = u_2 - S_2$, 并将其发送给 S_1
 - 8: S_1 计算 $V = t_1 + t_2$, 并生成它的补码二进制表示 $V^{(l-1)} LV^{(0)}$
 - 9: 对从 0 到 $(1-1)$ 的每一位 j , S_1 生成随机数 $V_1^{(j)} \in \phi_{2^l}$, 并计算 $V_2^{(j)} = V^{(j)} \oplus V_1^{(j)}$ S_1 将 $V_2^{(I-1)} LV_2^{(0)}$ 发送给 S_2
 - 10: S_1 和 S_2 共同计算 $\left(u_1^{(1-1)} \dots u_1^{(0)}, u_2^{(1-1)} \dots u_2^{(0)}\right) =$

$$\text{BitAdd} \left(v_1^{(1-1)} \dots v_1^{(0)}, V_2^{(1-1)} \dots v_2^{(0)}, r_1^{(1-1)} \dots r_1^{(0)}, r_2^{(1-1)} \dots r_2^{(0)}\right)$$
 - 11: S_i 返回值 $u_i^{(1-1)}$
-

3.4.3 前向传播过程

我们在服务器之间设置了一系列安全的交互协议, 如图14所示。这些协议应用于不同的层并成为前向学习的构建模块。

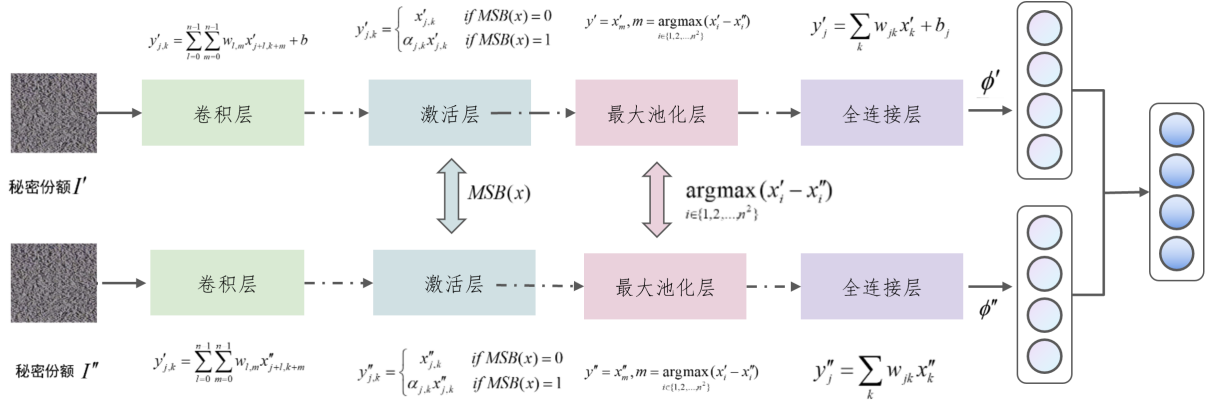


图 14: 参数生成模块

I. 卷积层

卷积层的卷积运算是在滤波器和输入的局部区域之间执行点积再求和。这个计算过程是可分布的，服务器 S_1 和 S_2 用公共权重 w 和偏差 b 在本地执行卷积层的前向传递 (计算)。

对于第 j 层的卷积层输出的第 t 通道的第 k 个神经元：

S_1 将计算神经元的一个分量：

$$y'_{j,k} = \sum_{l=0}^{n-1} \sum_{m=0}^{n-1} w_{l,m} x'_{j+l,k+m} + b_t$$

S_2 将计算神经元的另一个分量：

$$y''_{j,k} = \sum_{l=0}^{n-1} \sum_{m=0}^{n-1} w_{l,m} x''_{j+l,k+m}$$

其中，该层滤波器大小为 $n \times n$ ，神经元对应的的共享权重为 $w_{l,m}$ ($l, m = 0, 1, \dots, n-1$)，对应的偏置为 b_t ， $x_{j,k}$ 来表示第 j 层第 k 个神经元对应的输入激活。我们将 S_2 中所有的偏置都设为 0。

II. 全连接层

全连接层中的神经元与前一层中的所有激活都有完全连接，它们的激活可以通过矩阵乘法和偏置偏移来计算，偏置偏移也满足关联性和分布性。因此，基于我们的安全添加协议，全连接层的前向传递也可以在 S_1 和 S_2 中本地执行。具体来说，对于该层中的第 k 个神经元：

S_1 将计算输出的一个分量： $y'_k = \sum_i w_{i,k} x'_i + b_k$

S_2 将计算输出的一个分量： $y''_k = \sum_i w_{i,k} x''_i$

我们使用 x_i 来表示上一层的第 i 个神经元的激活输入, 使用 $w_{i,k}$ 表示上一层第 i 个神经元到当前第 k 个神经元连接的权重, b_k 表示当前层第 k 的神经元的偏置。我们仍然将 S_2 中的偏差设置为 0。

III. 激活层

激活层的目的是将非线性引入网络, 现在神经网络默认建议是使用 ReLU, ReLU 的主要缺点还是当 $x < 0$ [16], 它不能基于梯度的方法学习, 所以模型采用 leaky ReLU 函数来避免这一情况。

ReLU 层在卷积层和全连接层的输出处应用 $\max(x, \alpha x)$ 。在本模型中, 每个元素由 S_1 和 S_2 共享, 即 $x = x' + x''$, 通过应用上面的安全 MSB 协议, S_1 和 S_2 将共同确定 x 的最高有效位 MSB。如果 MSB 为 0, 我们有 $\max(x, \alpha x) = x$; 如果 MSB 为 1, $\max(x, \alpha x) = \alpha x$ 。对于上一层 (第 $j-1$ 层) 输入的第 k 个神经元, S_1 将计算:

$$y'_{j,k} = \begin{cases} x'_{j-1,k}, & \text{MSB}(x) = 0 \\ \alpha_{j,k} x'_{j-1,k}, & \text{MSB}(x) = 1 \end{cases}$$

同时, S_2 将计算:

$$y''_{j,k} = \begin{cases} x''_{j-1,k}, & \text{MSB}(x) = 0 \\ \alpha_{j,k} x''_{j-1,k}, & \text{MSB}(x) = 1 \end{cases}$$

IV. 池化层

池化层沿着空间维度执行向下采样操作, 常用的池化方式包括平均池化和最大池化。平均池化即将池化区域中所有值的平均值作为输出, 最大池化即为在池化区域中选择最大值进行输出。假设池化层步长均为 n , 每个 $n \times n$ 的池化区域中, 包含数字 $x_i (i \in \{1, 2, \dots, n^2\})$, 在我们的模型中, x_i 被分成了两份 x' 和 x'' , 分别被 $S_1 S_2$ 持有。

如果采用平均池化, y 是对应的池化区域输出, S_1 将计算:

$$y' = \text{avg}(x'_i) (i \in 1, 2, \dots, n^2)$$

S_2 将计算:

$$y'' = \text{avg}(x''_i) (i \in 1, 2, \dots, n^2)$$

如果采用最大池化, S_1 和 S_2 可以基于我们的 BitExtra 算法, 获得最大值的索引, 而

不显示实际值。对于池化区域中的 x_i 和 x_j ,

$$\begin{aligned}
(x_i < x_j) &= \text{MSB}(x_i - x_j) \\
&= \text{MSB}((x'_i + x''_i) - (x'_j + x''_j)) \\
&= \text{MSB}((x'_i - x'_j) + (x''_i - x''_j)) \\
&= \text{BitExtra}((x'_i - x'_j), (x''_i - x''_j))
\end{aligned}$$

如果输出为 0, 则 $x_i > x_j$, 如果输出为 1, 则 $x_i < x_j$ 。池化区域内共有 n^2 个数, 需要进行 $n^2 - 1$ 次这样的比较, S_1 和 S_2 可以确定最大值的索引。

对于 S_1 , 将输出:

$$y' = x'_m, m = \arg \max (x'_i - x''_i) \quad (i \in \{1, 2, \dots, n^2\})$$

对于 S_2 , 将输出:

$$y'' = x''_m, m = \arg \max (x'_i - x''_i) \quad (i \in \{1, 2, \dots, n^2\})$$

在上述图的展示中, 采用的是最大池化。

3.4.4 反向传播过程

反向传播是通过计算出输出层的误差, 从后往前逐层传递计算出所有层的误差, 最后求出该层误差对权重和偏置的梯度, 最后根据梯度下降法对权重和偏置进行更新。CNN 常用的损失函数 C 包含均方差与交叉熵, 损失函数 C 对输出层 a^J 的求导结果如下, 其中 y 为输入数据的理想输出, 即对应标签向量化。在后续的反向传播中会使用。

$$\frac{\partial C}{\partial A^J} = (A^J - Y)$$

假设梯度迭代步长为 α , 最大迭代次数 MAX, 停止迭代阈值为 ε 。每一层的误差为 δ , 激活函数输入为 z , 输出为 a 。当所有权重 w 和偏置 b 的变化小于阈值 ε 时, 停止迭代。

I. 输出层

反向传播过程先根据确定的损失函数, 确定输出层的误差, 记输出层 J 的误差为 δ^J ,

$$\delta^J = \frac{\partial C}{\partial z^J} = \frac{\partial C}{\partial a^J} \frac{\partial a^J}{\partial z^J} = (a^J - y) \cdot \sigma'(z^J)$$

其中, $\sigma'(z^J)$ 是激活函数的导函数, $\mathbf{e}$ 表示 Hadamard 积, 即对应逐元素相乘。双服务器各自持有理想输出 (标签) 的秘密份额 y', y'' , 对于第 k 个神经结点来说: S_1 计算:

$$\delta'_k = (a' - y') \times \sigma'(z_k)$$

S_2 计算:

$$\delta_k'' = (a'' - y'') \times \sigma'(z_k)$$

双方合成得到 δ_k 。

II. 全连接层误差传递

全连接层的误差传递公式:

$$\delta^j = \frac{\partial C}{\partial z^j} = \frac{\partial C}{\partial a^{l+1}} \frac{\partial z^{j+1}}{\partial z^j} = \delta^{j+1} \frac{\partial z^{j+1}}{\partial z^j}$$

对于第 j 层全连接层的第 k 个神经元的误差, 由它影响过的下一层神经元的误差计算得到。

对于第 k 个神经元, 进行如下计算:

$$\delta_{j,k} = \sum_{k'} \delta_{j-1,k'} w_{k'}$$

对于全连接层的参数更新, 输入的 m 张图片的对于该层的输入均参与。

$$S_1 \text{ 计算: } \Delta W' = \alpha \sum_{i=1}^m \delta_j (Y'_{i,j-1})^T$$

$$S_2 \text{ 计算: } \Delta W'' = \alpha \sum_{i=1}^m \delta_j (Y''_{i,j-1})^T$$

该层的权重矩阵 W 更新: $W = W - \Delta W' - \Delta W''$

偏置矩阵 B 的更新: $B = B - \alpha \sum_{i=1}^m \delta_i$

I. 卷积层的误差传递

对于第 j 层卷积层, 位于第 t 层通道的 (x, y) 的神经元的误差传递, 由它影响过的下一层神经元的误差计算得到。对应的误差计算:

$$\delta_j(x, y) = \sum_{x_m} \sum_{y_n} \delta_{j+1}(x_m, y_n) w(a, b) \sigma'(z^j(x, y))$$

其中, (x_m, y_n) 是被影响过的神经元, w 是对应的权重, σ' 是对应该神经元的激活函数的导数值。对于卷积层的参数更新, 输入的 m 张图片的对于该层的输入均参与。 S_1 计算: $\Delta W' = \alpha \sum_{i=1}^m Y'_{i,j-1} * \delta_{i,j}$ S_2 计算: $\Delta W'' = \alpha \sum_{i=1}^m Y''_{i,j-1} * \delta_{i,j}$ 对于权重矩阵 W 的更新: $W = W - \Delta W' - \Delta W''$ 对于偏置矩阵 B 的更新: $B = B - \alpha \sum_{i=1}^m \sum_{u,v} (\delta^{i,j})_{u,v}$

IV. 池化层的反向传播

池化层的误差传递据池化方式而定, 假设该池化层为所有层中的第 j 层。

使用平均池化时, 第 $j + 1$ 层的神经元是第 j 层相应池化区域的平均值, 则第 j 层池化区域每个结点的误差为对应第 $j + 1$ 层神经元误差除以池化面积。

使用最大池化时, 第 $j + 1$ 层的神经元是第 j 层相应池化区域的最大值, 则第 j 层池化区域对应的最大值结点误差为对应第 $j + 1$ 层神经元误差, 池化区域其他节点误差值为 0。

我们用 `upsample()` 来记录池化层传递误差的操作。

对于 S_1 : $\delta'_l = \text{upsample}(\delta'_{l+1})$

对于 S_2 : $\delta''_l = \text{upsample}(\delta''_{l+1})$

3.5 系统基础模块的实现方案

本节主要介绍本系统中用户系统相关的模块。用户系统是连接用户与最终训练服务器的关键, 包含用户访问控制, 用户数据管理, 项目管理, 日志记录等功能, 系统并不存储用户的数据, 因此可以在保障数据隐秘性的同时向用户隐藏具体细节。

3.5.1 用户管理模块

用户管理的实现方案如图15所示。

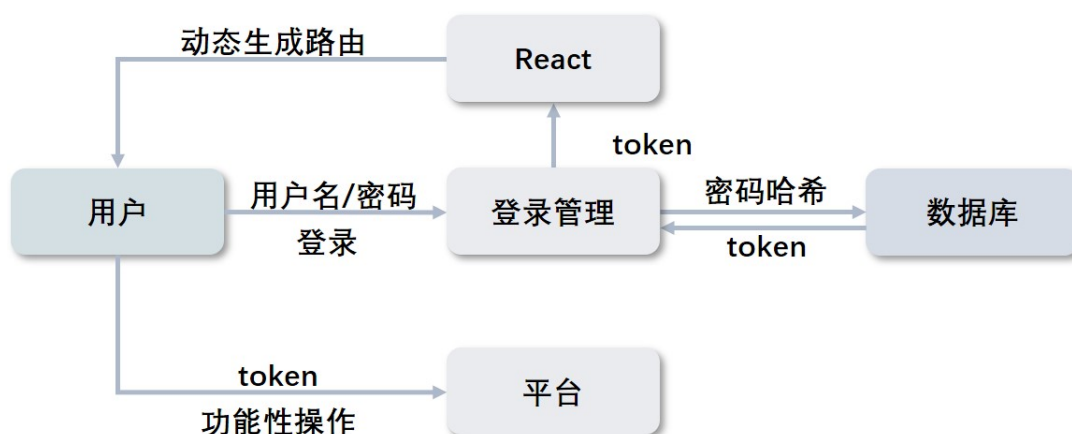


图 15: 用户管理实现方案

用户管理模块主要用来对用户进行管理, 不同的用户具有不同的权限。目前, 本系统将用户分为管理员用户和普通用户, 其中普通用户又根据不同的项目分为创建者和协作者。整个用户管理以 React 框架为基础进行实现。用户在登录时系统会根据数据库中记录的用户权限将对应的唯一用户标识符返回给前端, 前端会根据这一标识符设置不同的侧边栏, React 会针对不同的侧边栏会动态的生成路由, 没有权限的用户无法访

问相关路由。当用户需要进行功能上的操作，例如创建项目时，系统后端会根据用户的 React 来判断用户是否有权限，如果权限允许就允许该操作，否则返回错误。

3.5.2 数据管理模块

数据管理的实施方案如图16所示。

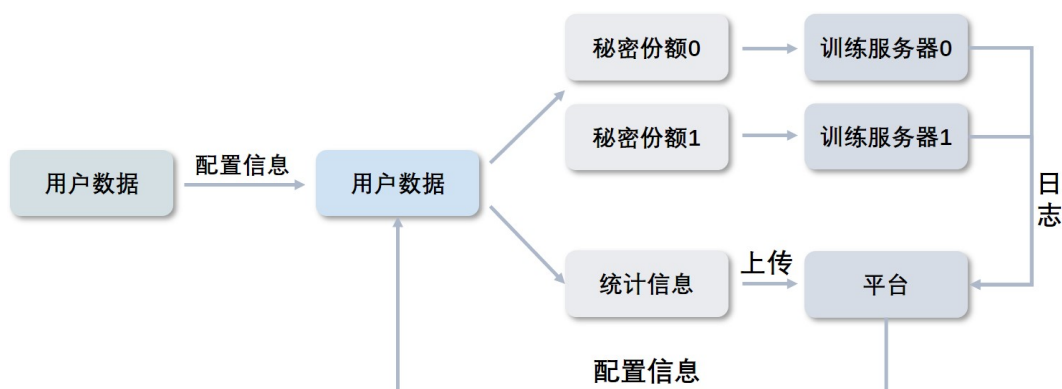


图 16: 数据管理实施方案

数据管理模块主要用来对用户上传的数据进行管理。为了保证用户数据的绝对保密，该模块并不保存数据的真实数据，而仅保存数据的统计信息以及对应数据集的散列值。为此我们编写了一款用于协助对用户数据进行秘密分享的开源软件 ShareHelper (分享助手)，该软件离线工作，可以自动识别用户选择的数据集，对数据集中数据大小之类的相关信息进行了统计，自动将用户的数据集进行秘密分享并存储在本地，自动计算数据的散列值。最终用户只需要上传 ShareHelper 产生的说明文件到系统即可，系统会自动将统计结果与数据集相关联。当用户将该数据集加入到某一项目后，当项目开始训练时，系统会自动生成上传接口，用户只需要按照提示将本地生成的两个文件依次传给两个训练服务器即可，服务器会自动校验数据集的散列值和统计信息。

3.5.3 项目管理模块

项目管理的实施方案如图17所示。

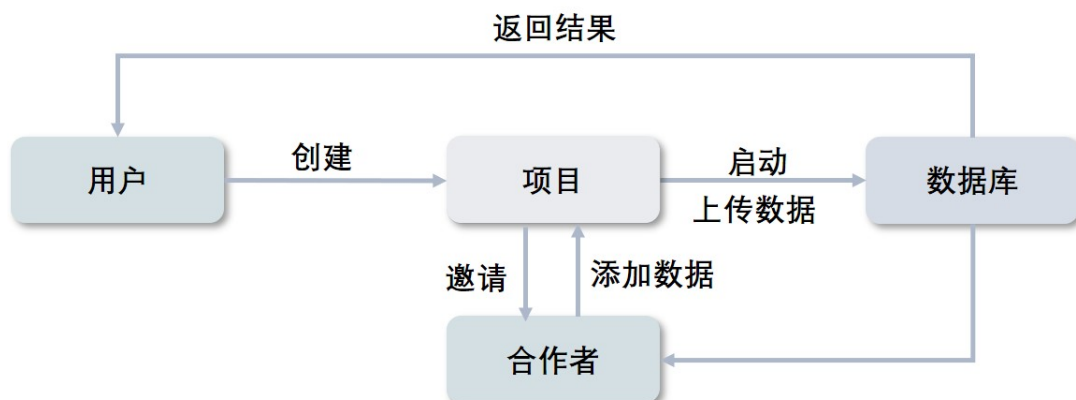


图 17: 项目管理实施方案

项目管理模块主要用来对用户创建的项目进行管理。在该模块中所有的操作都基于登录时的用户权限，普通用户只能对自己创建的项目进行编辑，只有项目的创建者和合作者才能查看该项目等。

一个项目的四种状态分别为准备中，训练中，暂停和完成，根据用户的操作，项目可以在四种状态中切换。我们使用数据库实时记录每个项目的状态变化。只有当项目不处于训练中状态时才可以进行数据的添加。当项目完成时系统提供模型在线预览和下载功能。开始训练前用户需要以项目为单位分别向服务器上传数据。

在项目的各个阶段，项目管理模块都会实时的记录每一个项目的各项操作请求，在训练正式开始前，系统会记录的信息包括用户发起的创建请求，邀请他人加入的请求，数据添加请求等一系列操作记录及其时间；在项目正式开始训练后会实时记录项目进度，包括每一轮模型的准确率，损失函数，训练时间等参数；在项目训练完成后记录项目产生的模型参数，在线模型等信息，记录项目的完成信息。

3.5.4 系统日志模块

系统日志模块的实现方案如图18所示。

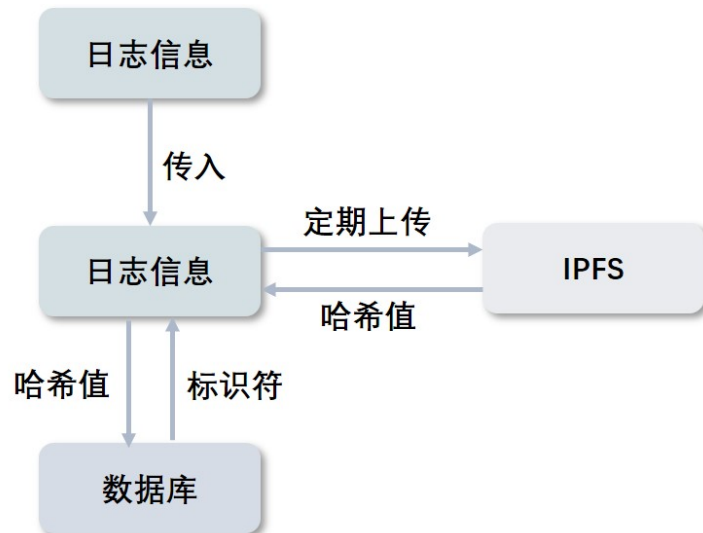


图 18: 系统日志实现方案

系统日志模块主要记录系统在用户使用过程中进行的各项关键操作，除了项目管理模块中提到的关于项目训练阶段所记录的各项日志之外还会对用户管理模块，数据上传模块等进行关键信息的记录，记录内容包含用户的创建，删除，用户信息的更新，用户数据的上传等。

为了保护系统的安全性，可监控性和操作记录的不可篡改性，本系统使用基于 IPFS 的日志系统，将上述章节中采集到的日志存储在星际文件系统中，通过 IPFS 的加密算法保证了其不可篡改和删除的特性。

本系统会定时的将所有记录下来的日志做一个打包，然后将打包的结果写入一个 IPFS 节点，然后获取并记录返回哈希值，将哈希值和对应的时间节点存入数据库中，数据库会返回对应的标识符。当管理员需要查看某段时间的系统日志时，只需要根据标识符，通过数据库读取这个值，然后就可以根据这个唯一的哈希值从 IPFS 文件系统上获取到相应的文件信息。

3.6 数据交换模块

数据交换模块主要用于在客户端和服务端之间，服务器和服务器之间进行数据的交换。在上传数据时，客户端和服务端之间需要进行数据交换，在进行数据隐藏和模型训练时，服务器之间需要进行大量的数据交互。这些参数均为关键数据或者关键参数，为了保证这些数据的安全，本系统使用安全套接字（Secure Socket Layer, SSL）协议保证数据的安全传输。

SSL 协议具有保密性，鉴别性和完整性，能够保证两个系统之间发送的所有敏感资料都不会读取和篡改。该模块的开发主要基于 Python 的 socket 模块，该模块提供了使用 SSL 协议进行通信的接口。我们使用开源软件包 OpenSSL 来生成证书并实现通信双方互相认证，以此来保证数据发送到正确的对象，并且防止数据被窃听或篡改。

使用 SSL 协议进行数据交换的流程如图19所示：

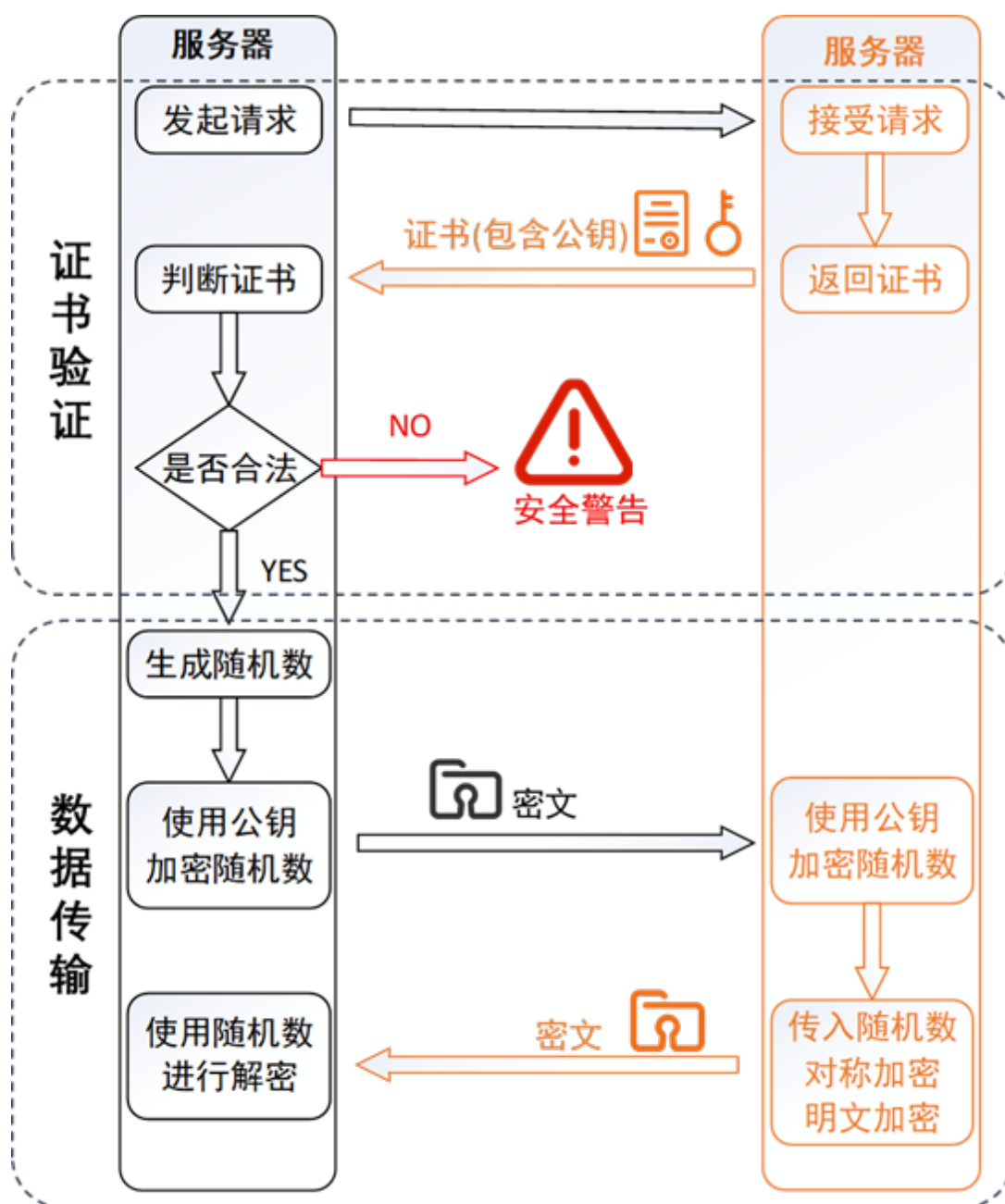


图 19: SSL 协议使用流程示意图

第四章 系统测试

系统测试主要是对本系统的功能和安全方面进行测试。功能方面的主要评判指标为双服务器模型神经网络相比单服务器神经网络运行时的时间和准确率。安全方面主要使用针对联邦学习的深度梯度泄露攻击对系统的模型训练进行攻击测试。同时还会对系统的基础功能进行测试，保证系统的可用性。

4.1 测试环境

测试主要围绕双服务器模型的多方安全计算进行测试，数据提供方由用户主机 1，用户主机 2，用户主机 3 组成，训练使用的服务器则由训练服务器 1 和训练服务器 2 组成。整个用户系统搭建在用户系统服务器上，使用内网穿透技术映射到公网作为中介进行使用。具体测试环境如表3所示。

表 3: 测试环境

设备名	硬件信息
用户主机 1	8g 内存 Intel(R) Core(TM) i5-8250U CPU
用户主机 2	12g 内存 Intel(R) Core(TM) i7-7500U CPU
用户主机 3	8g 内存 Intel(R) Core(TM) i7-7700HQ CPU
训练服务器 1	196g 内存 Intel(R) Xeon(R) Gold 6248 CPU
训练服务器 2	196g 内存 Intel(R) Xeon(R) Gold 6248 CPU
用户系统服务器	16g 内存 Intel(R) Core(TM) i5-7300HQ CPU

4.2 系统功能性测试

系统功能性测试细分为数据隐藏效果测试、数据隐藏性能测试、模型训练效果测试、模型训练速度测试。

4.2.1 数据隐藏效果测试

本系统的数据隐藏模块基于 Instahide 算法对图像数据进行隐藏，Instahide 算法在数据增强算法 Mixup 的基础上通过添加掩码等方法进一步加强了混合后数据的隐秘性。为了衡量混合后数据的隐秘性，我们使用结构相似性 (SSIM) [17] 来评判两张图片的相似程度。SSIM 由德州大学奥斯丁分校的图像和视频工程实验室提出，由亮度对

比、对比度对比、结构对比三部分组成，能够有效衡量两张图片的相似度指标。具体的测试设计如表4所示。

表 4: 数据隐藏效果测试

测试代号	测试 1-1
测试方法	将图片用 Instahide 算法进行处理，然后和原始图片进行 SSIM 指标的计算
测试目的	查看 Instahide 算法的隐蔽性能
测试项目	Instahide 算法处理后的图片和原始图片的 SSIM 指标
测试结果	混合前后图片数据的 SSIM 指标都比较低，可以接受

在具体测试中，我们随机选取 CIFAR 10 数据集中的 10000 张图片，将这 10000 张图片使用 Instahide 算法进行混合，共生成了 10000 张混合后的图片，我们将混合后的图片与原始图片用 SSIM 进行相似度衡量。为了方便进行展示，我们每 500 张图采集一个平均值，测试结果如图20所示。

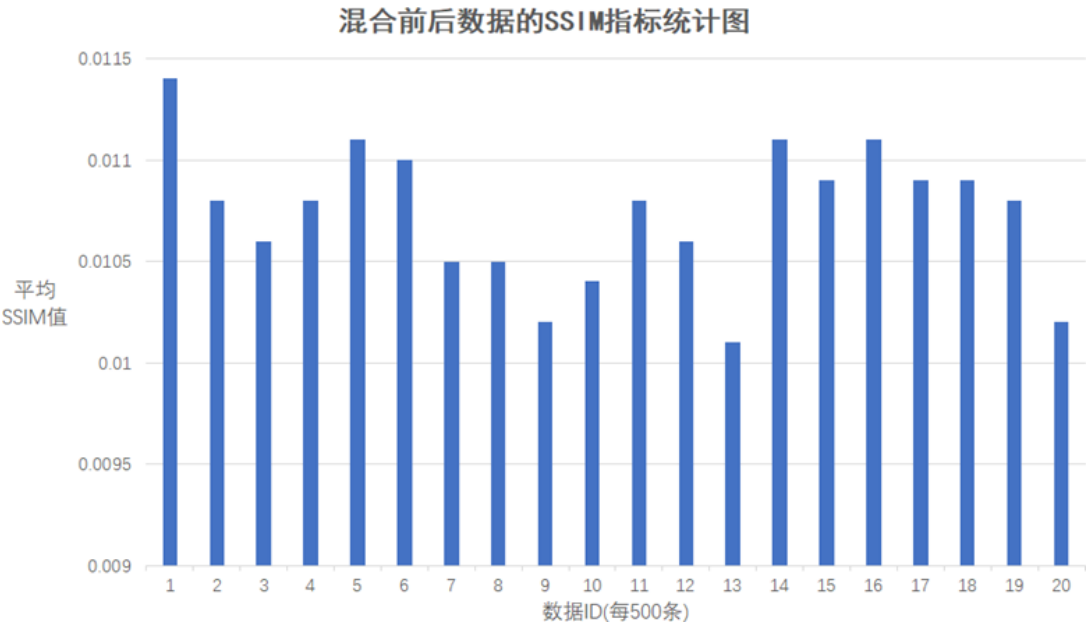


图 20: 图片混合前后 SSIM 指标统计图

从图20我们可以看到，在使用 Instahide 算法前后，图片的 SSIM 指标平均值的范围都在 0.01 0.012 这个范围区间，相对来说比较稳定，同时指标值很低，说明混合后的图像与原图的相似度非常低，很难从混合后的图像推算出原始图像。

4.2.2 数据隐藏性能测试

为了进一步保证用户信息的隐秘性，本系统的数据隐藏过程发生在两个服务器之间，需要两个服务器协同完成。相比于单服务器模型的数据隐藏过程，双服务器模型的

数据隐藏需要花费更多的时间，本节对双服务器共同完成 Instahide 的过程进行时间测试。具体测试数据如表5所示。

表 5: 数据隐藏效果测试

测试代号	测试 1-2
测试方法	分别用单服务器模型和双服务器模型测试 Instahide 混合不同数据量所需时间
测试目的	查看双服务器模型 Instahide 带来的性能下降能否接受
测试项目	Instahide 完成数据混合所需的时间
测试结果	双服务器模型和单服务器模型相比性能无明显下降

最终测试结果如图21所示。

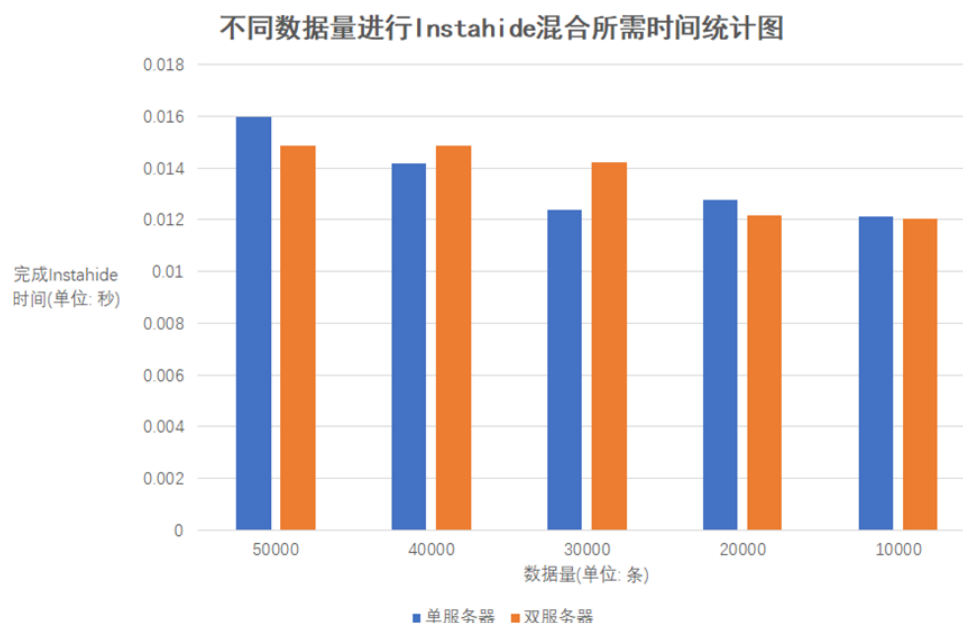


图 21: 不同数据量进行 Instahide 混合所需时间统计图

从图21可以看出，从总体趋势来看，数据量越多，完成 Instahide 所需的时间就越多。由于在实现上采用了 GPU 加速，由 GPU 完成向量的运算，因此总体所需时间较少，在一定范围内数据量的增加对所消耗的时间并无显著影响。由于双服务器模型在使用时需要消耗大量的网络带宽用来传输数据，该时间会随着网络状况的拥堵或畅通产生较大的变化，因此在进行时间记录时忽略数据交换的时间。从单服务器模型和双服务器模型的比较来看，双服务器模型做 Instahide 混合时性能稍有下降，但是下降不多，总体上消耗时间也和使用的数据量成反比。

4.2.3 模型训练效果测试

模型训练效果测试主要是为了测试在双服务器模型下，进行机器学习模型训练的性能测试，目的是保证在双服务器模型下，模型的准确率能够达到甚至超过单一服务器进行训练的准确率。在进行训练时，我们选择残差神经网络（ResNet18）作为测试模型。具体测试设计如表6所示。

表 6: 模型训练效果测试

测试代号	测试 1-3
测试方法	利用双服务器模型和单服务器模型用相同的数据集训练同一种神经网络，对比模型正确率
测试目的	双服务器模型下对机器学习模型的准确率能否达到较高水平
测试项目	服务器完成 20 轮训练后达到的准确率
测试结果	单服务器模型和双服务器模型训练效果基本一致

在我们的预测中，双服务器模型在进行训练时，仅仅会对训练时间产生一个较大的影响，双服务器模型和单服务器模型训练得到的机器学习模型准确率应该是相近的。具体的测试结果如下图22所示。

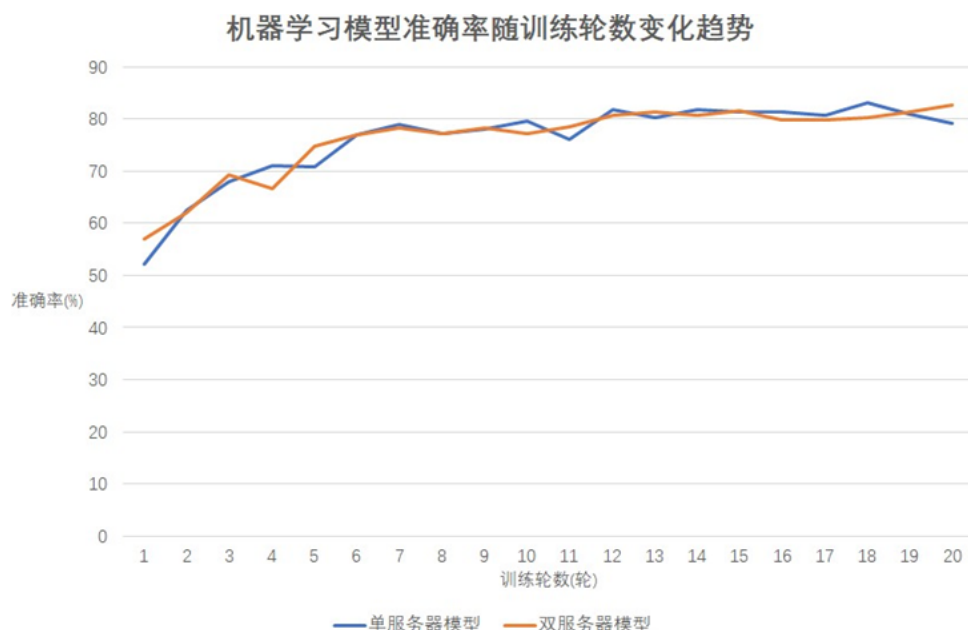


图 22: 机器学习模型准确率统计图

4.2.4 COT、同态加密计算速度比较

本系统在 Beaver 三元组生成中使用了 COT 技术，与基于同态加密的多方安全计算协议相比，有较大的速度提升。本节在相同隐秘计算任务、密钥强度的情况下，比较了

运用 COT 技术和运用同态加密完成计算所需要的时间。

受计算机性能所限，我们在 32bit 的密钥强度下，计算任务单位为 $32 \times 32 \times 3$ 的图片，大数模除运算使用模重复平方法进行优化的条件下，进行性能的比较。具体设计测试如表7所示。

表 7: 加密速度计算比较测试

测试代号	测试 1-4
测试方法	在 32bit 的密钥强度下，计算任务单位为 $32 \times 32 \times 3$ 的图片，查看计算完成的时间消耗
测试目的	比较相同计算任务下 COT 技术与同态加密技术的性能
测试项目	完成计算任务的时间
测试结果	COT 完成任务的时间相比同态加密有较大提升

从图23中可以看出，COT 完成计算任务的时间显著少于 LHE，约为 LHE 完成相同任务所需时间的一半，在任务量较少时，时间与任务量呈线性关系；当任务量较大时，由于受到 CPU 和内存的限制，计算耗时有部分增加。

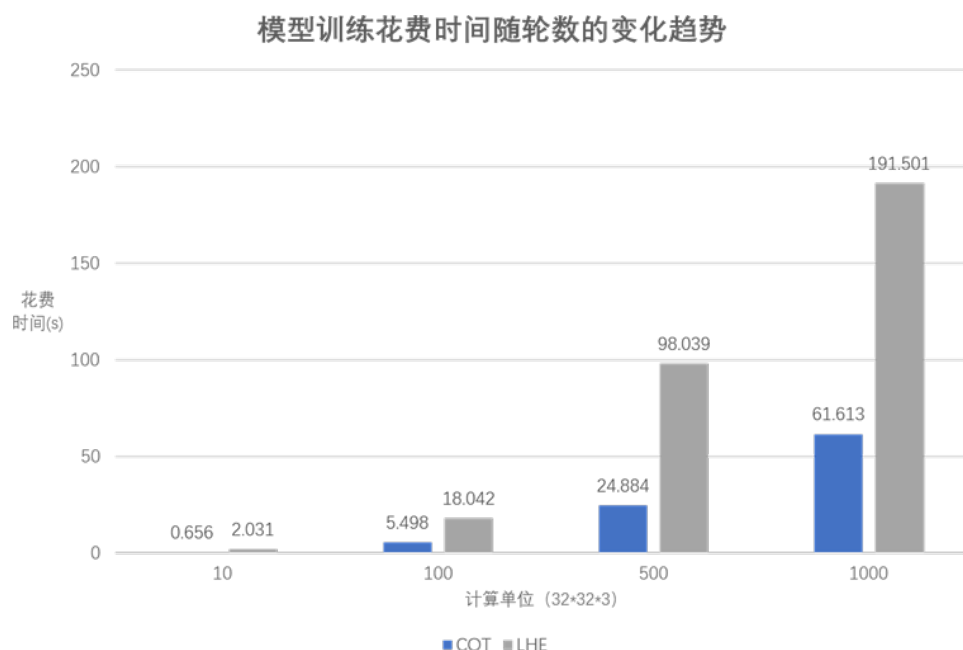


图 23: COT 和同态加密所需时间统计对比图

4.2.5 模型训练速度测试

模型训练速度测试主要是测试在双服务器模型下，最终机器学习模型的训练速度，测试该架构下的模型训练能否在达到较高准确率的同时也能保持一个较快的训练速度。具体测试设计如表8所示。

表 8: 模型训练速度测试

测试代号	测试 1-5
测试方法	利用双服务器模型和单服务器模型训练同一种神经网络，查看训练的时间消耗
测试目的	双服务器模型下对机器学习模型的训练速度能否满足要求
测试项目	服务器完成 20 轮训练所需的时间
测试结果	训练时间有所增加，但是可以接受

可以预见的是，由于双服务器模型在训练时，需要进行较多的参数交换，且参数的规模和输入的训练集的规模相同，因此这一阶段需要花费较多的时间，因此双服务模型的总机器学习模型训练时间会大大超出单服务器模型。具体测试数据如图24所示。

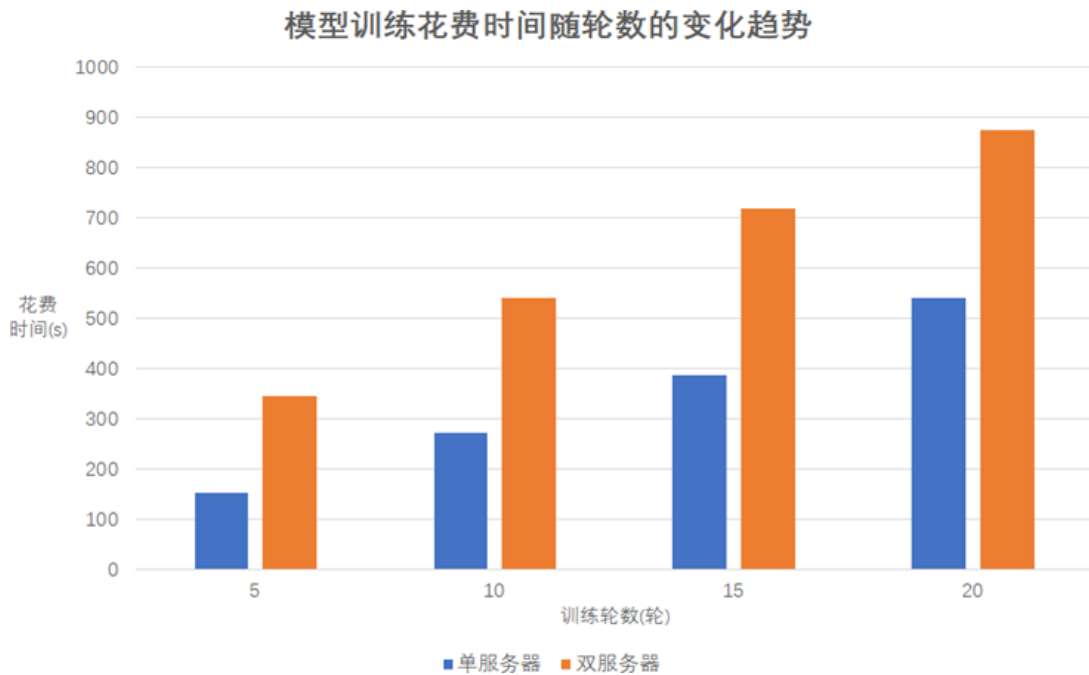


图 24: 双服务器模型和单服务器模型完成 100 轮训练所需时间统计图

从图24中可以看出相比于单服务器模型的模型训练时间，双服务器模型的训练时间有所增加，在训练 20 轮时，时间多出了 334s。双服务器模型在训练开始阶段需要大量的时间进行初始化，因此前 5 轮训练的时间会比后面 5-10 轮的训练的时间长。

4.3 系统安全性测试

本节主要使用深度梯度泄露 [18] 来对进行模型训练的服务器进行攻击。由于本系统在进行模型训练前先使用 Instahide 算法对用户的原始数据份额进行了混合，因此从输入模型进行训练的数据并不能推算出原始数据。具体测试方案如表9所示。

表 9: 模型安全性测试

测试代号	测试 1-6
测试方法	利用 DLG 攻击进行模型训练的服务器
测试目的	本系统的模型训练过程能否防御 DLG 攻击
测试项目	模型训练过程无法防御 DLG 攻击，但是攻击者无法获得原始数据
测试结果	和预期结果一致，DLG 攻击得到的图片是完全的噪音

为了更加直观的展示 DLG 攻击的效果以及本系统对该攻击防御的有效性，我们使用 CIFAR10 中的一张图片做可视化展示，效果如图25所示，25（a）是训练使用的原始图像，25（b）是直接使用该图像进行机器学习模型训练时，DLG 攻击恢复的原始图像，可以看到虽然在图像的左上角有几个噪点，但是图像的主体部分已经被还原出来，并且和原图像基本一致。图25（c）则是经过 Instahide 加密后再使用本系统进行机器学习模型训练的图片受到 DLG 攻击的效果，可以看到整张图片都是乱码，没有可辨识的部分。

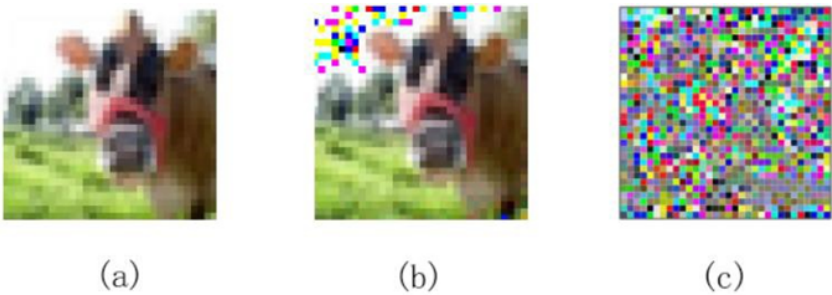


图 25: DLG 方法攻击效果图

4.4 系统完整性测试

本节主要对整个系统的完整性进行测试，从创建用户开始，先后使用系统的项目创建功能，数据上传功能，邀请合作功能，模型训练功能，在模型训练完成后使用系统提供的接口对模型进行测试。我们对整个系统的使用流程进行测试，保证系统、服务器、客户端能够正常通信并完成相应的任务。具体测试方案如表9所示。

表 10: 系统完整性测试

测试代号	测试 1-7
测试方法	模拟新用户，完成注册、登录、身份管理等操作
测试目的	本系统功能是否完善，能否顺利完成任务
测试项目	系统能够正常完成训练任务，模型结果正常
测试结果	结果符合预期，用户能够按照系统指引顺利完成多方安全计算

下面对测试到的模块进行展示。

4.4.1 用户注册测试

用户注册主要是往系统中新增一个具有某种权限的使用者，并将用户信息写入数据库。在注册用户时，输入用户姓名、邮箱地址和密码，点击“**Register**”按钮，完成用户注册。

用户注册界面如图28所示。

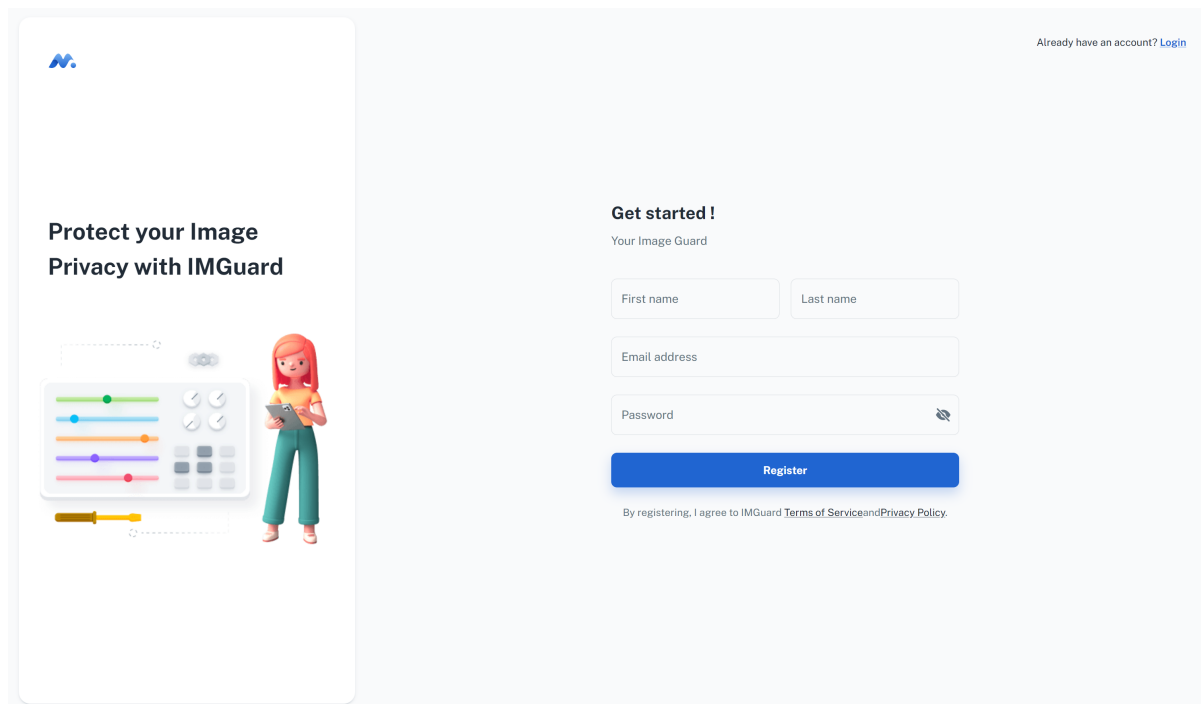
The image shows a user registration interface for a service called IMGuard. On the left, there is a promotional graphic with the text "Protect your Image Privacy with IMGuard" and an illustration of a woman holding a smartphone next to a control panel with various sliders and buttons. On the right, the registration form is titled "Get started !" and "Your Image Guard". It includes a link "Already have an account? Login" in the top right corner. The form fields are: "First name" and "Last name" (two separate input boxes), "Email address" (one input box), and "Password" (one input box with a toggle for visibility). A blue "Register" button is positioned below the password field. At the bottom of the form, there is a line of text: "By registering, I agree to IMGuard Terms of Service and Privacy Policy."

图 26: 用户注册界面

在注册界面右上角，提示已有账户的用户直接登录。

4.4.2 用户登录测试

用户登录是用户进入系统的入口，登陆时用户输入用户名和密码，同时系统将用户登录信息写入日志服务器，同时在数据库中查找用户身份记录，如果匹配则允许登录。

用户登录界面如图27所示。

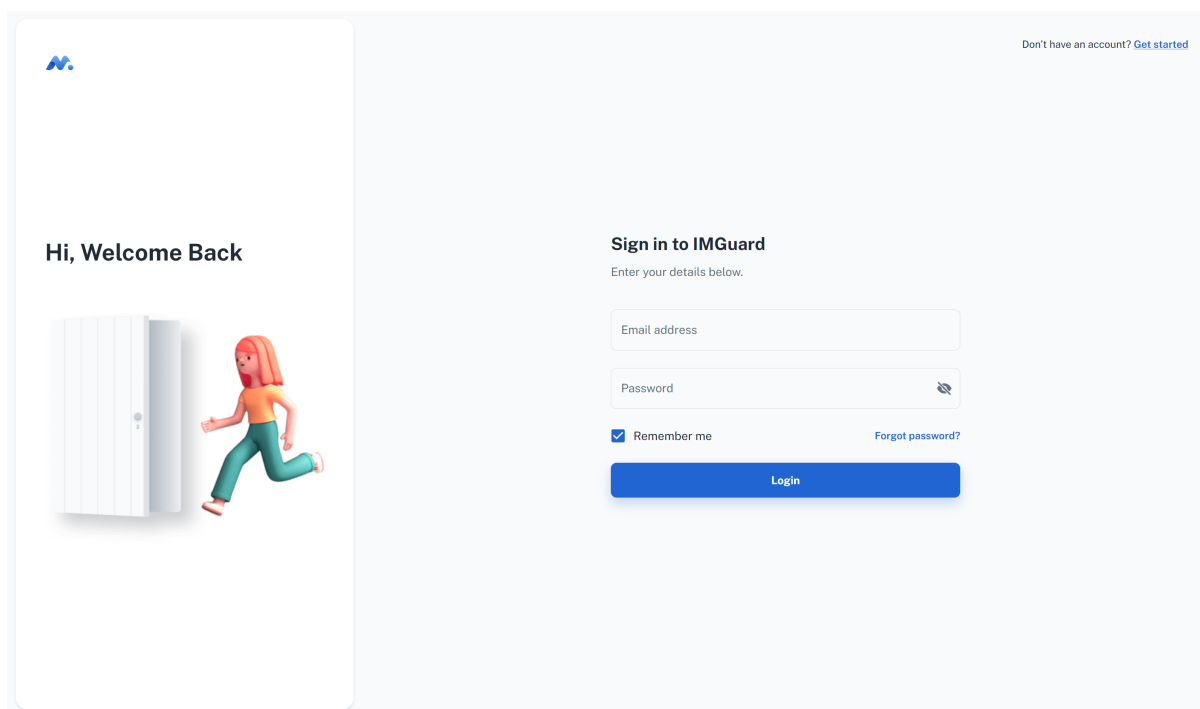


图 27: 用户登录界面

在登录界面，用户可以选择记住密码；此外，如果用户忘记密码，可进行密码找回。

4.4.3 用户管理测试

用户管理模块主要用来对用户进行访问权限控制，使得不同的用户具有对系统的不同访问权限。目前，本系统将用户分为管理员用户和普通用户，管理员用户可以查看系统日志，监控平台流量，查看所有用户创建的项目，对用户进行管理等。管理员的用户管理界面如用户注册主要是在系统的数据库中添加用户记录，不同的用户在系统中会使用不同的用户名进行标识，用户名由用户注册时设置。

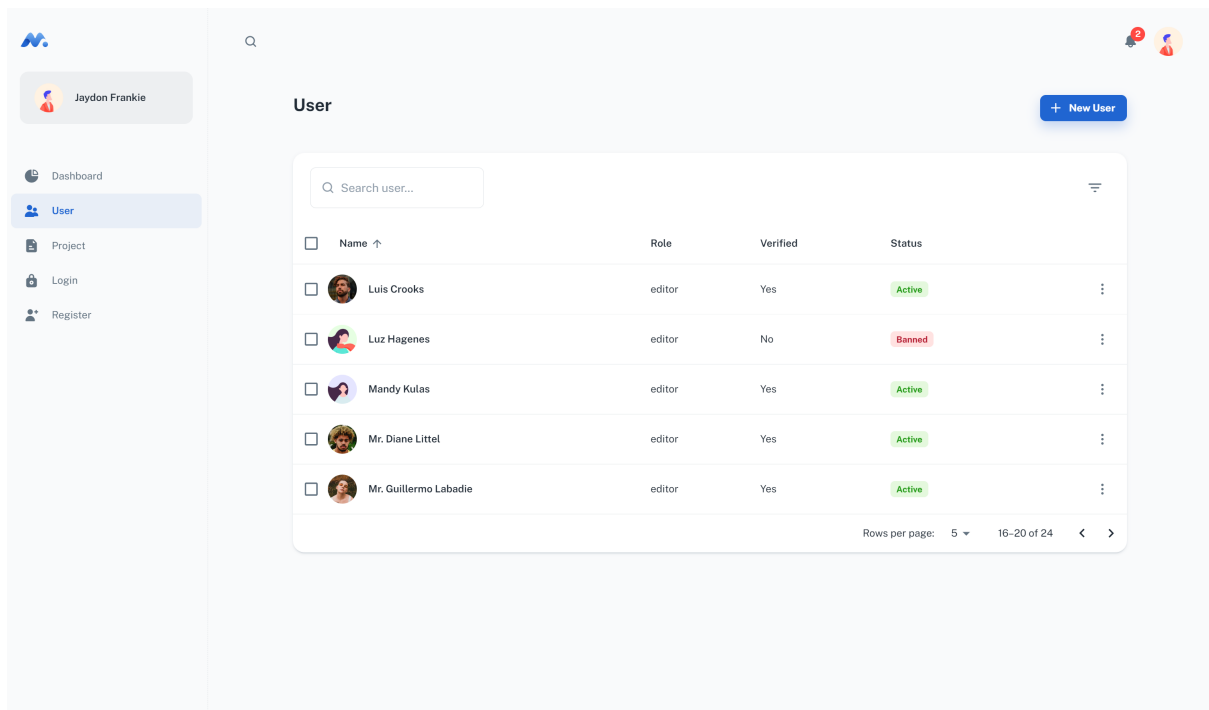


图 28: 用户管理界面

管理员可以在此进行用户的添加和权限管理。

4.4.4 项目管理测试

项目管理模块主要用来对用户创建的项目进行管理。用户可以在此页面进行项目的增删查改，还可以启动项目，暂停项目等。项目管理界面如图29所示。

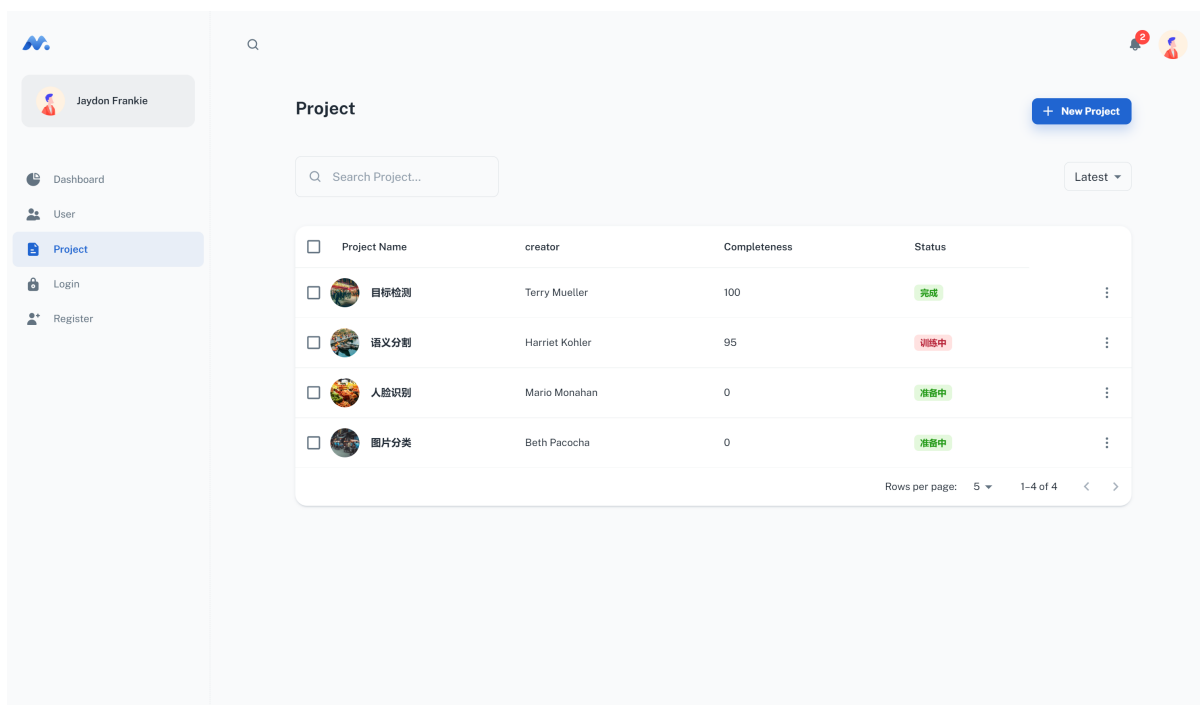


图 29: 项目管理模块展示

在发起项目时，编辑者需要输入项目的名称，为该项目添加描述信息，并上传训练数据集。只有当用户为管理者时，才可以进行项目创建。项目创建界面如图??所示。

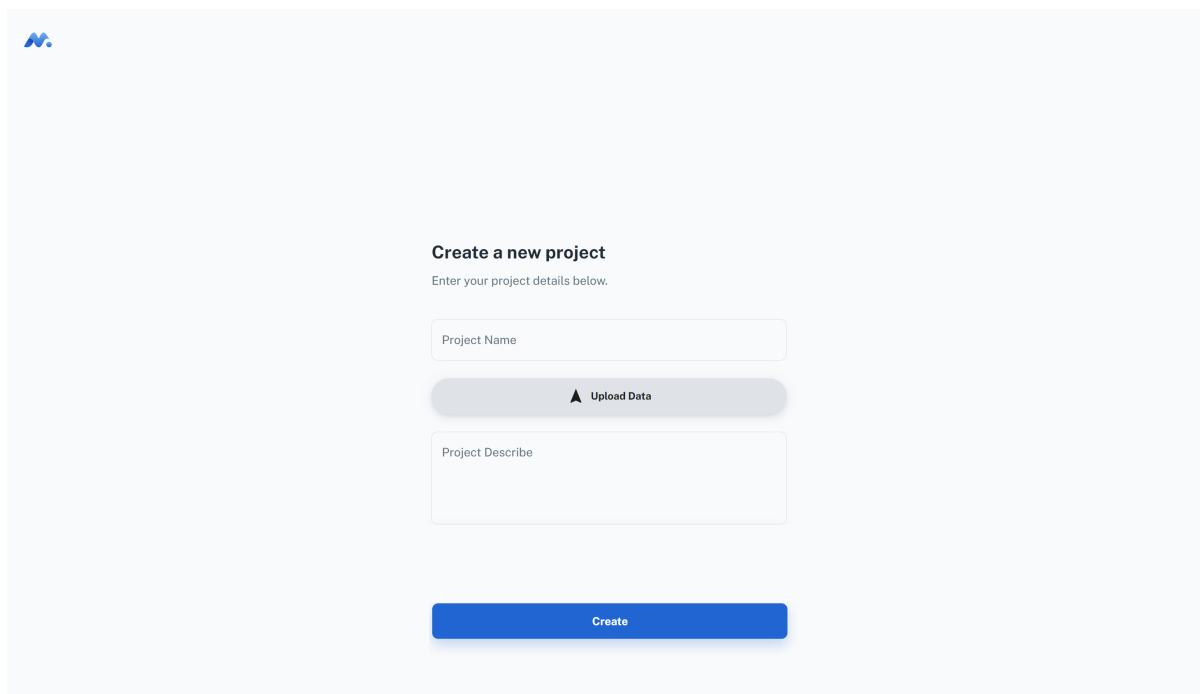


图 30: 创建项目

4.4.5 日志模块测试

系统日志模块主要记录系统在用户使用过程中进行的各项关键操作。为了保护系统的安全性，可监控性和操作记录的不可篡改性，本系统使用基于 IPFS 的日志系统，定期将上述章节中采集到的日志存储在星际文件系统中，将记录唯一地址的哈希值记录在数据库中。

为了方便管理员对系统进行实时的监控，在系统中增加了日志展示模块。在此模块中，管理员可以查看系统用户和任务信息，包括当前系统中的用户数量、创建的任务数量、正在训练的任务以及已经训练完成的任务；可以查看双服务器的实时负载（内存占用信息）以及日志服务器的实时负载信息；可以快速查看近期创建的任务名和任务创建的时间，据此还可以路由到任务详情页面。

日志展示界面如图31所示。



图 31: 日志模块

第五章 创新性

本章节主要基于本系统的两个核心模块——双服务器 Instahide 数据隐藏算法和双服务器神经网络模型，阐述了系统的创新点：

5.1 赋能数据安全

本系统采用双服务器 Instahide 数据实例隐藏技术，确保了图像数据的存储安全性和训练安全性。针对明文传输方式存在的图像泄露风险，使用份额划分和 Mixup 数据增强的数据混合方法，防范系统被攻击之后所导致的数据泄露问题；针对模型在密文域内学习和运算的困难性，通过 MPC 改进后的 Instahide 算法对图像进行实例隐藏，完成数据加密的同时也保证了图像的可训练、可推断性；针对图像窃取后被恶意利用的问题，在训练前对用户传入的原始数据进行简单、高效的加密，系统只存储加密后的图像，确保在神经网络训练的过程中，即使数据被攻击者窃取、恢复，也无法还原出原始数据从而对图像数据进行了全方位的保护。

5.2 提供可靠算法

本系统使用基于双服务器的分布式训练架构。目前常规的神经网络训练中，对受训练数据的隐私保护是一个普遍薄弱的环节。恶意攻击者不但能够通过最后的训练结果轻易推导出训练数据，甚至可以无需最后的结果，而仅通过在训练过程中传递的梯度参数就可以对他人的训练数据进行反向恢复，这对用户的数据安全是一个不小的潜在隐患。针对多方神经网络训练中的隐私保护问题，本项目设计了一种双服务器的神经网络训练协议。首先对用户输入图像通过 InstaHide 实例隐藏方法实现用户图像数据的数据混合和加密技术；然后双服务器基于 MPC 改进后的 CNN 算法对图像数据进行学习、处理，从而完成对图像的安全应用，保证了即使在神经网络训练过程不能证明其安全性的前提下，恶意攻击者也无法仅通过攻击训练过程还原出用户上传的原始数据，提供当前分布式框架中的增强安全功能。同时，经过加密后的图像仍然具有训练能力，本系统基于 MPC 实现了对加密数据的训练算法，并取得了优良的性能。相对于同态加密，Instahide 实例隐藏方法可以达到速度更快、存储量更少、精度损失小，这有助于多源的大数据处理，也适应深度学习发展对于数据的要求。

5.3 打通数据孤岛

本系统提出基于双服务器的神经网络训练协议，数据所有者可将他们的私有数据分布在两个非协作服务器合作进行特征提取以及模型训练。我们利用秘密共享技术完成两个服务器的交互，同时保证神经网络的准确性和数据的隐私安全性。本系统提供的可靠的数据协作训练系统，提高了共享数据隐私保护的安全性、在保护所有者数据隐私的情况下，大大提升了不同来源数据集共享的可能，打通了数据孤岛，让原本不敢被共享的数据得以流通，打破数据壁垒，赋能多方协作、驱动业务创新。

5.4 分割数据权属

现有的图像保护主要是依靠出台政策来规范管理数据，但该方法存在很多的弊端，如企业遵守程度、政府监管力度，这些都属于是人为因素，无法保障，因此即使制定了如此多的相关政策，图像隐私泄露问题仍然是一个难以整治的问题。而本系统从图像的角度出发，直接对图像进行保护处理，实现数据权属的明确。系统只具有图像的使用权，而不具有所有权，只有所属用户拥有图像的所有权。通过是分离数据所有权与使用权，不交易数据本身，只交易数据的计算结果，让数据交易与价值核算合理化。在保证图像数据应用价值的同时，避免了企业滥用数据，企业中存储图像被窃取等隐私泄露风险。

第六章 总结

本系统通过加法秘密共享、Beaver 乘法三元组、相关忘性传输等技术，设计了双服务器模型下的用户数据 Instahide 协议以及双服务器模型下的卷积神经网络训练协议。双服务器模型的引入，使得用户在对系统零信任的前提下，仍能进行安全的将数据输入模型训练，解决了用户使用隐私图像数据进行神经网络训练时易泄露的问题，同时很大程度上规避了单台服务器受攻击所带来的用户图像隐私泄露风险；同时，在数据训练前通过 Instahide 协议对数据进行混淆，能够抵御如梯度泄露攻击等以还原数据集为目的的攻击，大大增强了多用户在使用隐私数据集进行协同训练时的安全性，从而保护了人脸隐私。

未来，我们希望将本系统发展为一个可扩展的图像安全应用系统，能有更多的图像数据处理算法和训练算法能够在本系统支持的框架上正常使用，在保证图像数据隐私安全的同时，充分实现图像价值的应用；同时，将双服务器模型进行扩展，使用更加安全的多服务器模型进行算法的改进，不仅仅局限于 CNN 等简单模型，也可以应用于 Transformer 等更加复杂的模型，从而提高本系统功能的安全性、复杂性以及运算效率。

本系统拥有广阔的应用前景。数据时代的来临对私有财产中的图像数据提出了更高的保护需求，不仅要求避免图像未经授权而被窃取，同时，如何在保护图像数据隐私的同时不影响训练的准确性，也是一个迫切的具有现实意义的难题，而本系统针对此问题提供了一种可行的解决方法。此外，本系统适用于任何图像隐私数据合作的应用背景，比如医疗行业中的疾病预测、金融业的行情预测、社会学的舆情分析以及科研中的数据合作等，凡是需要训练图像隐私数据的场景，系统都能够为其提供保证其数据安全性的合作机会；对于提高数据的利用率、促进神经网络发展、加强安全和隐私保护方面具有重大优势和意义。但安全性的保证同样带来了计算的冗余，这也同样是制约高安全性密码算法商用的重要问题，如何设计高效而安全的计算协议，是当前也是未来数据安全领域的重要技术难题。如何快速、安全地对数据进行分发；如何在对密态的数据进行高效计算，是未来隐私计算研究的一大重要方向。借助本系统提出的技术手段，可以有效地支持医疗、金融、保险等隐私数据较多的行业安全地使用数据进行计算，对促进数据流通、发掘数据潜力有重要意义。

参考文献

- [1] 唐春明, 胡业周. 基于多比特全同态加密的安全多方计算[J]. 计算机学报, 2021, 44(4): 836-845.
- [2] 沈蒙, 张杰, 祝烈煌, 等. 面向征信数据安全共享的 SVM 训练机制[J]. 计算机学报, 2021, 44(4): 696-708.
- [3] BRAKERSKI Z, HALEVI S, POLYCHRONIADOU A. Four round secure computation without setup[C]//Theory of Cryptography Conference. Springer, 2017: 645-677.
- [4] CLEAR M, MCGOLDRICK C. Multi-identity and multi-key leveled fhe from learning with errors[C]//Annual Cryptology Conference. Springer, 2015: 630-656.
- [5] MUKHERJEE P, WICHS D. Two round multiparty computation via multi-key fhe[C]//Annual International Conference on the Theory and Applications of Cryptographic Techniques. Springer, 2016: 735-763.
- [6] HUANG K, LIU X, FU S, et al. A lightweight privacy-preserving cnn feature extraction framework for mobile sensing[J]. IEEE Transactions on Dependable and Secure Computing, 2019, 18(3): 1441-1455.
- [7] HUANG Y, SONG Z, LI K, et al. Instahide: Instance-hiding schemes for private distributed learning[C]//International Conference on Machine Learning. PMLR, 2020: 4507-4518.
- [8] ZHAO C, ZHAO S, ZHAO M, et al. Secure multi-party computation: theory, practice and applications[J]. Information Sciences, 2019, 476: 357-372.
- [9] LI J, KUANG X, LIN S, et al. Privacy preservation for machine learning training and classification based on homomorphic encryption schemes[J]. Information Sciences, 2020, 526: 166-179.
- [10] ARCHER D W, BOGDANOV D, LINDELL Y, et al. From keys to databases—real-world applications of secure multi-party computation[J]. The Computer Journal, 2018, 61(12): 1749-1771.
- [11] MRAZOVA I, KUKACKA M. Image classification with growing neural networks[J]. International Journal of Computer Theory and Engineering, 2013, 5(3): 422.
- [12] HE K, ZHANG X, REN S, et al. Spatial pyramid pooling in deep convolutional networks for visual recognition [J]. IEEE transactions on pattern analysis and machine intelligence, 2015, 37(9): 1904-1916.
- [13] KELLER M, ORSINI E, SCHOLL P. Mascot: faster malicious arithmetic secure computation with oblivious transfer[C]//Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. 2016: 830-842.
- [14] PAILLIER P. Public-key cryptosystems based on composite degree residuosity classes[C]//International conference on the theory and applications of cryptographic techniques. Springer, 1999: 223-238.
- [15] CHOU T, ORLANDI C. The simplest protocol for oblivious transfer[C]//International Conference on Cryptology and Information Security in Latin America. Springer, 2015: 40-58.
- [16] LECUN Y, BENGIO Y, HINTON G. Deep learning[J]. nature, 2015, 521(7553): 436-444.
- [17] WANG Z, BOVIK A C, SHEIKH H R, et al. Image quality assessment: from error visibility to structural similarity[J]. IEEE transactions on image processing, 2004, 13(4): 600-612.
- [18] HITAJ B, ATENIESE G, PEREZ-CRUZ F. Deep models under the gan: information leakage from collaborative deep learning[C]//Proceedings of the 2017 ACM SIGSAC conference on computer and communications

- security. 2017: 603-618.
- [19] ZHU L, LIU Z, HAN S. Deep leakage from gradients[J]. Advances in Neural Information Processing Systems, 2019, 32.
- [20] 唐春明, 胡业周, 李习习. 基于多密钥全同态加密方案的无 CRS 模型安全多方计算[J]. 密码学报, 2020, 8(2): 273-281.
- [21] ADAMS S, MELANSON D, DE COCK M. Private text classification with convolutional neural networks [C]//Proceedings of the Third Workshop on Privacy in Natural Language Processing. 2021: 53-58.
- [22] HELBITZ I, AVIDAN S. Reducing relu count for privacy-preserving cnn speedup[J]. arXiv preprint arXiv:2101.11835, 2021.
- [23] HUANG Y, GUPTA S, SONG Z, et al. Evaluating gradient inversion attacks and defenses in federated learning[J]. Advances in Neural Information Processing Systems, 2021, 34: 7232-7241.